

Performance of a vertex-centred block-coupled finite volume methodology for small-strain static elastoplasticity

Federico Mazzanti^{a,b,c,d} , Philip Cardiff^{a,b,c,d} ,*

^a School of Mechanical and Materials Engineering, University College Dublin, Dublin 4, Ireland

^b SFI I-Form Centre Advanced Manufacturing Research Centre, University College Dublin, Dublin 4, Ireland

^c Bekaert University Technology Centre, School of Mechanical and Materials Engineering, University College Dublin, Dublin 4, Ireland

^d University College Dublin Centre for Mechanics, University College Dublin, Dublin, Ireland

ARTICLE INFO

Dataset link: <https://doi.org/10.5281/zenodo.15497350>

Keywords:

Finite volume method
Solid mechanics
Small-strain elastoplasticity
Vertex-centred
Block-coupled
Newton-Raphson

ABSTRACT

This article assesses the performance of a vertex-centred block-coupled finite volume methodology for static small-strain elastoplasticity based on a Newton-Raphson algorithm. The proposed coupled solution algorithm is compared with vertex-centred and cell-centred segregated solution procedures in terms of accuracy, efficiency and robustness for quasi-static problems. This coupled methodology is then verified on four test cases: a patch test to check its ability to reproduce first-order polynomials; the method of manufactured solutions to check the order of convergence of displacement and stress; a 2-D perforated plate and a 3-D narrow T-member under elastoplastic conditions to examine the accuracy of the solutions on non-trivial test cases. The proposed methodology demonstrates high accuracy and a computational efficiency of up to 28× speedups over the standard cell-centred segregated approach.

1. Introduction

The finite volume method offers several variations tailored to solving static elastoplastic problems, which can be categorised by solution algorithm — either *implicit* or *explicit* — or grid arrangement, such as cell-centred or vertex-centred [1]. Within the scope of solution algorithms, implicit formulations can be further divided into three primary approaches: segregated, block-coupled and matrix-free methods. The *segregated* approach decouples the governing momentum equation into individual scalar equations, applying outer Picard or Fixed-Point iterations to reintroduce the necessary coupling. In contrast, *block-coupled* methods (or simply “coupled” methods) discretise the momentum equation in a vectorised form, thereby handling inter-component coupling within a larger, denser system matrix for more direct interaction among components. *Matrix-free* approaches differ significantly by completely avoiding the assembly of system matrices; instead, they typically augment the governing equation with an inertia term and advance the solution in pseudo-time. *Fully explicit* approaches operate similarly but advance the solution in real physical time, without the need for matrix assembly. In terms of memory efficiency, matrix-free and explicit methods have the smallest memory requirements, followed by implicit segregated approaches. Coupled methods, however, generally demand more memory due to the larger system matrix. In terms of computational times, the trend is typically reversed for quasi-static problems, where coupled approaches might be expected to more quickly achieve convergence. Explicit methods, however, offer an advantage in handling stronger non-linearities as they

* Corresponding author at: School of Mechanical and Materials Engineering, University College Dublin, Dublin 4, Ireland.
E-mail address: philip.cardiff@ucd.ie (P. Cardiff).

avoid the need for iterative convergence. Additionally, stability is a key factor when choosing between implicit and explicit methods. Explicit schemes are conditionally stable and subject to the Courant–Friedrichs–Lewy condition, which can significantly restrict the time step size and limit their efficiency for long-term simulations. In contrast, implicit methods are typically unconditionally stable, making them more suitable for quasi-static problems, where they often converge faster despite higher memory and computational demands.

In terms of grid arrangement, *cell-centred* [2–38] and *vertex-centred* [39–52] approaches are the most widely used finite volume formulations in solid mechanics, though alternative configurations such as *face-centred* [53] and *staggered* [54] arrangements are also viable. Cell-centred approaches are advantageous in that they avoid the construction of an additional, or *dual*, grid often required in vertex-centred methods. However, they lack computational nodes on the boundaries, which can complicate the implementation of coupled solutions [24]. Cell-centred approaches simplify the discretisation of volume integrals, as the computational nodes typically align with the centroid of the control volume. Conversely, vertex-centred methods more easily allow the implementation of implicit boundary conditions due to the availability of nodes on the domain boundary. Vertex-centred approaches typically employ finite-element-style shape functions, which constrain the types of permissible cells, although alternative formulations without shape functions have also been developed [50,55,56]. In addition, an intermediate class of methods, commonly referred to as *vertex-to-cell* approaches, has also been introduced in the literature [57]. These methods retain vertex-based degrees of freedom while circumventing the need for dual mesh construction by computing fluxes directly over the surrounding control volumes.

Significant advancements have been achieved in developing finite volume methods for static elastoplastic problems. However, a comprehensive comparative analysis of the available solution formulations is still lacking in the literature. The primary objective of this study is to address this gap by systematically comparing the following finite volume solution strategies: vertex-centred coupled and segregated strategies, as well as cell-centred segregated methodologies in terms of performance and accuracy, specifically for static elastoplastic problems. Matrix-free or explicit approaches present another viable option for comparison; however, their performance is not analysed in this work.

A review of the relevant literature highlights the evolution of finite volume formulations for elastoplasticity. Pioneering contributions to segregated procedures can be attributed to Demirdžić and co-workers [2,3], whose work laid the foundation for elastoplastic analysis. Maneeratana [11] extended these methods to accommodate large-strain deformations, incorporating both total and updated Lagrangian frameworks. Subsequently, Demirdžić et al. [12] expanded the scope to address hygro-thermo-elastic problems, while Jasak and Weller [13], Tuković et al. [20], Bijelonja et al. [26] and Sclaro et al. [33] focused on linear elasticity. Further advancements include Cardiff et al. [21], who investigated orthotropic elasticity and Bijelonja et al. [14,16], who addressed hyperelasticity. Bašić et al. [15] made notable contributions to the development of segregated solvers for viscoelasticity. Tang et al. [22] developed a segregated implementation for poro-elastoplasticity, while Cardiff et al. [25] and Whelan et al. [58] advanced the application of segregated procedures to metal forming and damage processes.

Despite its advantages, such as simplicity of implementation and low memory requirements, the segregated algorithm often suffers from slow convergence, particularly in cases involving strong inter-component coupling. An alternative to segregated procedures is the block-coupled approach, which solves all unknowns simultaneously within a single matrix system, thereby improving the handling of inter-component coupling. Cardiff et al. [24] conducted a comparative analysis between the segregated and block-coupled algorithms for linear elasticity. Their study introduced a linear cell-centred block-coupled solver inspired by the framework established by Das et al. [18]. The results demonstrated significant improvements in convergence rates, leading to substantial reductions in computational times. Furthermore, their comparison with a commercial finite element code highlighted superior efficiency in terms of both time and memory requirements.

Block-coupled procedures have been applied across a variety of studies. Fallah et al. [43] proposed a vertex-centred finite volume framework for large elastic deformations, utilising a “modified” Newton-Raphson method. Taylor et al. [47] extended this framework to model von Mises plasticity, albeit retaining an approach similar to finite element methods by employing shape functions. González Acedo et al. [28] applied block-coupled procedures to thermo-elasticity, while Nungaray et al. [52] utilised control volume function approximation techniques and polynomial interpolators for managing surface and volume integrals in a similar context. Further advancements include Pimenta and Marcondes [51], who developed a coupled vertex-centred solver drawing parallels to the approach by Taylor et al. [47], wherein shape functions are defined for each cell. Clancy [30] introduced a pressure-displacement finite volume formulation that coupled pressure and displacement; however, this method only included pressure-displacement coupling in a coupled manner and treated inter-displacement-component coupling using a deferred correction strategy, which may not fully leverage the advantages of a truly block-coupled system.

This article presents an analysis of a vertex-centred, displacement-based finite volume algorithm, examining its computational memory requirements, runtime efficiency, accuracy and robustness. The investigation specifically targets von Mises plasticity under the assumption of infinitesimal strains, providing comparisons against vertex-centred and cell-centred segregated finite volume approaches. A key difference between the proposed approach and previous methods is the usage of a least squares method to approximate the gradients at the dual faces, avoiding the explicit requirement for shape functions. The proposed algorithm has been implemented within *solids4foam* – a toolbox for performing solid mechanics and fluid-solid interaction simulations in the open-source software OpenFOAM – and the code and cases are made publicly available on GitHub at www.github.com/federico081997.

The remainder of the article is constructed as follows: Section 2 outlines the governing equation and the constitutive equations of small-strain von Mises elastoplasticity; Section 3 presents the finite volume discretisation of the surface and body forces, as well as the solution procedure and the boundary conditions; Section 4 analyses the developed coupled solver on four test cases in order to assess its accuracy, efficiency and robustness in comparison with vertex-centred and cell-centred segregated solvers; finally, in Section 5 a summary of the results is outlined as well as future directions of the current work.

2. Mathematical model

In this section, the mathematical equations that describe the conservation of linear momentum for von Mises elastoplastic solids are presented. The formulation assumes infinitesimal strains and the absence of strain hardening.

2.1. Governing equations

The conservation of linear momentum for a quasi-static arbitrary body with volume Ω and surface Γ is given in integral form as:

$$Res(\mathbf{u}) = \underbrace{\oint_{\Gamma} \mathbf{n} \cdot \boldsymbol{\sigma}(\mathbf{u}) d\Gamma}_{\text{Surface forces}} + \underbrace{\int_{\Omega} \mathbf{F}_b d\Omega}_{\text{Body forces}} = 0, \quad (1)$$

where $\mathbf{n}$ is the outward-facing surface normal unit vector, $\boldsymbol{\sigma}$ is the engineering stress tensor and $\mathbf{F}_b$ is an external body force. The residual $Res(\mathbf{u})$ is zero for a displacement field $\mathbf{u}$ which satisfies the equation.

2.2. Constitutive equations

According to the small-strain elastoplastic theory, the total strain $\boldsymbol{\varepsilon}$ can be divided into elastic and plastic components:

$$\boldsymbol{\varepsilon} = \boldsymbol{\varepsilon}_e + \boldsymbol{\varepsilon}_p, \quad (2)$$

where subscripts e and p refer to elastic and plastic components, respectively. Considering this relation, the stress $\boldsymbol{\sigma}$ can be expressed in terms of the strain components as:

$$\boldsymbol{\sigma} = 2\mu(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_p) + \lambda \operatorname{tr}(\boldsymbol{\varepsilon} - \boldsymbol{\varepsilon}_p)\mathbf{I}, \quad (3)$$

or in terms of the displacement gradient, $\nabla \mathbf{u}$, as:

$$\boldsymbol{\sigma} = \mu \nabla \mathbf{u} + \mu \nabla \mathbf{u}^T + \lambda \operatorname{tr}(\nabla \mathbf{u})\mathbf{I}, \quad (4)$$

where $\mathbf{I}$ is the identity tensor. The first and second Lamé parameters λ and μ can be obtained as functions of Poisson's ratio, ν , and Young's modulus of elasticity, E .

$$\mu = \frac{E}{2(1+\nu)}, \quad (5)$$

$$\lambda = \frac{\nu E}{(1+\nu)(1-2\nu)}. \quad (6)$$

In the absence of strain hardening, the von Mises yield criterion can be expressed as:

$$f(\boldsymbol{\sigma}) = \|\operatorname{dev}(\boldsymbol{\sigma})\| - \sqrt{\frac{2}{3}}\sigma_y \leq 0, \quad (7)$$

where σ_y is the yield stress. When the yield function returns a value less than zero, it signifies that the material remains within the elastic regime. Conversely, when the yield function reaches a value of zero, it indicates the onset of plastic behaviour in the material.

The associative flow rule takes the form:

$$\Delta \boldsymbol{\varepsilon}_p = \varphi \frac{\partial f(\boldsymbol{\sigma})}{\partial \boldsymbol{\sigma}}, \quad (8)$$

where $\Delta \boldsymbol{\varepsilon}_p$ is the plastic strain increment and φ is the plastic multiplier, which is governed by the Kuhn-Tucker loading/unloading conditions:

$$\dot{\varphi} \geq 0, \quad (9)$$

$$\dot{\varphi} f(\boldsymbol{\sigma}) = 0, \quad (10)$$

$$\dot{\varphi} \frac{\partial f(\boldsymbol{\sigma})}{\partial \boldsymbol{\sigma}} = 0. \quad (11)$$

Combined with the consistency condition, the stress at the next state can be calculated using the radial return algorithm [59], as outlined in Algorithm 1.

3. Numerical method

The finite volume discretisation of the integral governing equation (equation (1)) is now presented. Section 3.1 describes the definition of the dual cells associated with the primary control volumes. Subsequently, Section 3.2 outlines the discretisation of the governing equation using the Newton-Raphson algorithm under the assumption of infinitesimal strains, as well as the proposed

Algorithm 1 Radial return-mapping algorithm for small-strain elastoplasticity, where superscript $t + 1$ indicates the new time level and t indicates the previous one.

-
- 1: **Calculate** deviatoric strain $\mathbf{e}^{t+1} = \boldsymbol{\epsilon}^{t+1} - \frac{1}{3}\text{tr}(\boldsymbol{\epsilon}^{t+1})\mathbf{I}$.
 - 2: **Calculate** deviatoric trial stress $\mathbf{s}_{\text{trial}}^{t+1} = 2\mu(\mathbf{e}^{t+1} - \boldsymbol{\epsilon}_p^t)$.
 - 3: **Calculate** yield function $f_{\text{trial}}^{t+1} = \|\mathbf{s}_{\text{trial}}^{t+1}\| - \sqrt{\frac{2}{3}}\sigma_y$.
 - 4: **if** $f_{\text{trial}}^{t+1} \leq 0$ **then** ▷ Elastic step.
 - 5: **Set** plastic strain $\boldsymbol{\epsilon}_p^{t+1} = \boldsymbol{\epsilon}_p^t$.
 - 6: **Update** stress $\boldsymbol{\sigma}^{t+1} = \kappa\text{tr}(\boldsymbol{\epsilon}^{t+1})\mathbf{I} + \mathbf{s}_{\text{trial}}^{t+1}$.
 - 7: **else** ▷ Plastic step.
 - 8: **Calculate** plastic return direction $\mathbf{n}_r^{t+1} = \frac{\mathbf{s}_{\text{trial}}^{t+1}}{\|\mathbf{s}_{\text{trial}}^{t+1}\|}$.
 - 9: **Calculate** plastic multiplier $\varphi = \frac{f_{\text{trial}}^{t+1}}{2\mu}$.
 - 10: **Update** plastic strain $\boldsymbol{\epsilon}_p^{t+1} = \boldsymbol{\epsilon}_p^t + \varphi\mathbf{n}_r^{t+1}$.
 - 11: **Update** stress $\boldsymbol{\sigma}^{t+1} = \kappa\text{tr}(\boldsymbol{\epsilon}^{t+1})\mathbf{I} + \mathbf{s}_{\text{trial}}^{t+1} - 2\mu\varphi\mathbf{n}_r^{t+1}$.
 - 12: **end if**
-

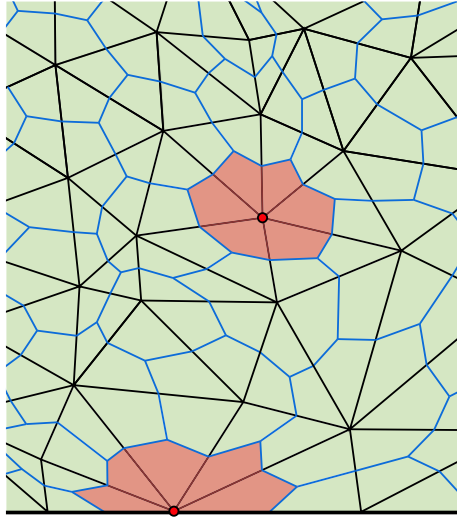


Fig. 1. Vertex-centred spatial discretisation on an unstructured triangular grid. The shaded areas represent internal and boundary computational control volumes (adapted from Hassan [50]).

least squares gradient implementation. Section 3.3 describes the overall solution procedure and the convergence criteria. Finally, Section 3.4 outlines the boundary conditions and how they are treated in the matrix system.

3.1. Solution domain discretisation

The solution domain is discretised into a finite number of non-overlapping convex polyhedral cells that collectively approximate and fill the computational domain. While this discretisation completely fills the computational domain used in the numerical model, it should be noted that, for curved geometries, the representation involves a geometric approximation, as planar-faced polyhedral cells cannot exactly conform to curved boundaries. However, this discrepancy reduces to zero as the mesh is refined. In the vertex-centred approach, an additional *dual* mesh is created, where each dual cell is constructed around a primary mesh vertex. Illustrated in Fig. 1, the construction of the dual mesh in a 2-D grid involves connecting a cell's centroid to the midpoint of its edges and then extending these connections to the centroid of adjacent cells. In a 3-D context, this process entails linking the cell's centroid to the centroid of the faces that lie between two neighbouring cells, and the edge midpoints. Notably, in the vertex-centred approach the vertices defining the dual cells, may not align with the geometric centres of the dual cells.

In this work, the vertex-centred approach is employed for the spatial discretisation primarily for two reasons. Firstly, the positioning of computational nodes at the boundaries facilitates the direct application of boundary conditions to these nodes, which can significantly simplify the matrix system that needs to be solved. Secondly, an additional advantage specific to tetrahedral meshes is that the cell-to-vertex ratio generally falls between 5 and 7 [60,61]. This leads to a reduced number of computational nodes in vertex-centred approaches for tetrahedral meshes compared to cell-centred methods, thereby significantly lowering the computational overhead associated with these meshes.

3.2. Discretisation procedure and solution algorithm

This section presents the vertex-centred finite volume discretisation framework developed in this work, detailing the discretisation of surface and body forces, along with the proposed least squares method for gradient approximation. Equation (1) is applied to each cell in the dual mesh, surrounding a primary mesh vertex.

3.2.1. Newton-Raphson solution algorithm

The Newton-Raphson method allows equation (1) to be iteratively solved, in terms of a sequence of linear system solutions and solution corrections:

$$\begin{aligned} \mathcal{J}(\mathbf{u}_n) \delta \mathbf{u}_n &= -\text{Res}(\mathbf{u}_n), \\ \mathbf{u}_{n+1} &= \mathbf{u}_n + \delta \mathbf{u}_n, \\ n &= 0, 1, \dots \end{aligned} \quad (12)$$

where $\mathcal{J}(\mathbf{u}_n) = \frac{\partial \text{Res}(\mathbf{u})}{\partial \mathbf{u}} \Big|_{\mathbf{u}=\mathbf{u}_n}$ is the Jacobian matrix evaluated at the current displacement field $\mathbf{u}_n$, $\delta \mathbf{u}_n$ is the displacement correction at iteration n , and $\text{Res}(\mathbf{u}_n)$ is the residual of the discretised momentum equation evaluated at iteration n .

3.2.2. Discretisation of the surface forces

Noting that the body force term in equation (1) is not a function of displacement, the contribution to the Jacobian for a dual cell around a primary mesh vertex can be expressed as

$$\begin{aligned} \mathcal{J} &= \frac{\partial}{\partial \mathbf{u}} \oint_{\Gamma} \mathbf{n} \cdot \boldsymbol{\sigma} \, d\Gamma \\ &= \frac{\partial}{\partial \mathbf{u}} \sum_{f=1}^{N_f} \int_{\Gamma_f} \mathbf{n} \cdot \boldsymbol{\sigma} \, d\Gamma_f, \end{aligned} \quad (13)$$

where the closed surface integral is represented as the cumulative sum of the surface integrals across all dual faces of the dual cell, and N_f refers to the number of faces of the dual cell.

An approximation is then introduced where it is assumed that the term $\mathbf{n} \cdot \boldsymbol{\sigma}$ exhibits a linear variation across each dual face. This assumption facilitates the application of the *midpoint rule* to approximate the face integrals as:

$$\mathcal{J} \approx \frac{\partial}{\partial \mathbf{u}} \sum_{f=1}^{N_f} \mathbf{n}_f \cdot \boldsymbol{\sigma}_f |\Gamma_f|, \quad (14)$$

where subscript f indicates a quantity at the centre of a dual face.

Generalising the procedure of Das et al. [19], the Jacobian $\mathcal{J}$ for a primary vertex can be expressed as

$$\mathcal{J}_{ij}^{\text{row,col}} = \frac{\partial}{\partial u_j^{\text{col}}} \sum_{f=1}^{N_f} (\sigma_{mi} \Gamma_m)_f^{\text{row}} \quad (15)$$

$$= \sum_{f=1}^{N_f} \frac{\partial (\sigma_{mi} \Gamma_m)_f^{\text{row}}}{\partial u_j^{\text{col}}} \quad (16)$$

$$= \sum_{f=1}^{N_f} \frac{\partial (\sigma_{mi} \Gamma_m)_f^{\text{row}}}{\partial G_{kl}^{\text{row}}} \left(\frac{\partial G_{kl}^{\text{row}}}{\partial u_j^{\text{col}}} \right)_f, \quad (17)$$

where index notation is adopted to deal more easily with third- and fourth-order tensors. The second order tensor $G_{kl} = \nabla \mathbf{u} = \partial u_k / \partial x_l$ represents the displacement gradient at a dual face f . Furthermore, the superscripts row and col refer to the rows and columns of the full Jacobian matrix. In this way, $\mathcal{J}_{ij}^{\text{row,col}}$ represents the 3×3 block coefficient in 3-D (or 2×2 in 2-D) located at (row, col) in the block matrix. Equation (17) can be further decomposed using the chain rule:

$$\mathcal{J}_{ij}^{\text{row,col}} = \sum_{f=1}^{N_f} \left(\Gamma_m \frac{\partial \sigma_{mi}}{\partial G_{kl}} + \frac{\partial \Gamma_m}{\partial G_{kl}} \sigma_{mi} \right)_f^{\text{row}} \left(\frac{\partial G_{kl}^{\text{row}}}{\partial u_j^{\text{col}}} \right)_f \quad (18)$$

$$= \sum_{f=1}^{N_f} (\Gamma_m C_{mikl} + \mathcal{G}_{mkl} \sigma_{mi})_f^{\text{row}} \left(\frac{\partial G_{kl}^{\text{row}}}{\partial u_j^{\text{col}}} \right)_f, \quad (19)$$

where the fourth-order tensor C_{mikl} denotes the *material tangent* at a dual face f , and the third-order tensor $\mathcal{G}_{mkl}$, known as the *geometric stiffness*, represents the sensitivity of the dual cell geometry to the displacement field (i.e., geometric non-linearities). Under

the assumption of small deformations, the geometric stiffness can be neglected because the surface area remains independent of the displacement. Consequently, equation (19) simplifies to:

$$J_{ij}^{\text{row,col}} = \sum_{f=1}^{N_f} (\Gamma_m C_{mikl})_f^{\text{row}} \left(\frac{\partial G_{kl}^{\text{row}}}{\partial u_j^{\text{col}}} \right)_f. \quad (20)$$

In this work, the material tangent in equation (20) is obtained numerically by using the finite difference approximation outlined in Algorithm 2.

Algorithm 2 Finite difference algorithm for the material tangent.

```

1: for all components of  $G_{kl}$  do
2:   Perturb component of  $G_{kl}$  by a perturbation amount  $h$ .
3:   Calculate perturbed stress  $\tilde{\sigma}_{mi}$  using Algorithm 1.
4:   Calculate components of  $C_{mikl}$  as  $(\tilde{\sigma}_{mi} - \sigma_{mi})/h$ .
5: end for

```

In Algorithm 2 the value of the perturbation h is chosen as a compromise between maximising the accuracy of the finite difference approximation (smaller h) and minimising round-off error (larger h). In this work, the perturbation parameter h is selected as the square root of machine precision, which is approximately 10^{-16} , multiplied by a representative value of the perturbed field. Given that the displacement gradient is typically in the order of 10^{-2} , the corresponding perturbation parameter h is set to 10^{-10} . The derived fourth-order material tangent C_{mikl} encompasses a total of 81 components. Nevertheless, due to the symmetry of the stress tensor and considering equation (4), the derivatives of the shear stresses with respect to the shear displacement gradients are identical for components related by swapping indices. For example, $\partial\sigma_{xy}/\partial G_{xy} = \partial\sigma_{xy}/\partial G_{yx}$, $\partial\sigma_{xy}/\partial G_{xz} = \partial\sigma_{xy}/\partial G_{zx}$ and so on. Consequently, this symmetry reduces the total number of distinct components to 36, allowing C_{mikl} to be stored as the following 6×6 matrix for each dual face:

$$C_{mikl} = \begin{bmatrix} C_{xxxx} & C_{xxyy} & C_{xxzz} & C_{xxxy} & C_{xxyz} & C_{xxzx} \\ C_{yyxx} & C_{yyyy} & C_{yyzz} & C_{yyxy} & C_{yyyz} & C_{yyzx} \\ C_{zzxx} & C_{zzyy} & C_{zzzz} & C_{zzxy} & C_{zzyz} & C_{zzzx} \\ C_{xyxx} & C_{xyyy} & C_{xyzz} & C_{xyxy} & C_{xyyz} & C_{xyzx} \\ C_{yzxx} & C_{yzyy} & C_{yzzz} & C_{yzxy} & C_{yzyz} & C_{yzzx} \\ C_{zxxx} & C_{zxyy} & C_{zxzz} & C_{zxxy} & C_{zxyz} & C_{zxzx} \end{bmatrix}, \quad (21)$$

where the subscripts indicate the specific components of the stress and displacement gradient being referenced.

3.2.3. Least squares gradient calculation method

A least squares procedure is proposed to compute the displacement gradient at the dual faces by fitting a plane through the displacement values at the primary vertices in each primary cell. This is in contrast to the vertex-centred finite volume methods in the literature [39–44,47,51] where finite-element-type shape functions are used to describe the displacement variation within primary cells. A potential advantage of the proposed least squares approach is that it is agnostic to the underlying cell shape, and is equally applicable to any convex polyhedra. In the case that a primary cell is a tetrahedron in 3-D (or triangle in 2-D), the proposed least squares fit is exact, and the displacement variation corresponds exactly to linear shape functions.

The least squares fit within each primary cell is performed as follows. Consider a primary cell with N_p primary vertices. To approximate the displacement gradient, $\mathbf{G}$, over this data, the least squares method minimises the sum of the squared residuals between the displacements and their corresponding fitted values.

The fitted plane can be described by the following equation:

$$\mathbf{u}(x, y, z) = \mathbf{a}x + \mathbf{b}y + \mathbf{c}z + \mathbf{d}, \quad (22)$$

where $\mathbf{a}$, $\mathbf{b}$ and $\mathbf{c}$ are the displacement gradient vectors along the x -, y - and z -directions, respectively, and $\mathbf{d}$ represents the displacement offset of the fitted plane. To achieve the best fit, the following least squares equation, denoted as $S(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d})$, must be minimised:

$$S(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}) = \sum_{p=1}^{N_p} [u_p - (\mathbf{a}x_p + \mathbf{b}y_p + \mathbf{c}z_p + \mathbf{d})]^2. \quad (23)$$

In the current implementation, the displacement offset vector, $\mathbf{d}$, is explicitly determined as the average displacement within each primary cell rather than treated as an unknown variable. This approach simplifies the system by avoiding the computational complexity of a 4×4 matrix, thereby improving the efficiency of the methodology. Thus, the displacement offset vector, $\mathbf{d}$, is calculated as:

$$\mathbf{d} = \frac{1}{N_p} \sum_{p=1}^{N_p} \mathbf{u}_p. \quad (24)$$

To minimise equation (23), the first derivatives of $S(\mathbf{a}, \mathbf{b}, \mathbf{c}, d)$ with respect to $\mathbf{a}$, $\mathbf{b}$ and $\mathbf{c}$ are set to zero:

$$\frac{\partial S(\mathbf{a}, \mathbf{b}, \mathbf{c}, d)}{\partial \mathbf{a}} = -2 \sum_{p=1}^{N_p} x_p [u_p - (ax_p + by_p + cz_p + d)] = 0, \quad (25)$$

$$\frac{\partial S(\mathbf{a}, \mathbf{b}, \mathbf{c}, d)}{\partial \mathbf{b}} = -2 \sum_{p=1}^{N_p} y_p [u_p - (ax_p + by_p + cz_p + d)] = 0, \quad (26)$$

$$\frac{\partial S(\mathbf{a}, \mathbf{b}, \mathbf{c}, d)}{\partial \mathbf{c}} = -2 \sum_{p=1}^{N_p} z_p [u_p - (ax_p + by_p + cz_p + d)] = 0. \quad (27)$$

Rearranging these equations in terms of $\mathbf{a}$, $\mathbf{b}$ and $\mathbf{c}$ leads to the formulation of the following coupled system:

$$\begin{bmatrix} \sum_{p=1}^{N_p} x_p^2 & \sum_{p=1}^{N_p} x_p y_p & \sum_{p=1}^{N_p} x_p z_p \\ \sum_{p=1}^{N_p} x_p y_p & \sum_{p=1}^{N_p} y_p^2 & \sum_{p=1}^{N_p} y_p z_p \\ \sum_{p=1}^{N_p} x_p z_p & \sum_{p=1}^{N_p} y_p z_p & \sum_{p=1}^{N_p} z_p^2 \end{bmatrix} \begin{bmatrix} \mathbf{a} \\ \mathbf{b} \\ \mathbf{c} \end{bmatrix} = \begin{bmatrix} \sum_{p=1}^{N_p} x_p (u_p - d) \\ \sum_{p=1}^{N_p} y_p (u_p - d) \\ \sum_{p=1}^{N_p} z_p (u_p - d) \end{bmatrix}. \quad (28)$$

The solution to this system is hence given by:

$$\mathbf{G} = \mathbf{M}^{-1} \mathbf{B}, \quad (29)$$

where,

$$\mathbf{M} = \begin{bmatrix} \sum_{p=1}^{N_p} x_p^2 & \sum_{p=1}^{N_p} x_p y_p & \sum_{p=1}^{N_p} x_p z_p \\ \sum_{p=1}^{N_p} x_p y_p & \sum_{p=1}^{N_p} y_p^2 & \sum_{p=1}^{N_p} y_p z_p \\ \sum_{p=1}^{N_p} x_p z_p & \sum_{p=1}^{N_p} y_p z_p & \sum_{p=1}^{N_p} z_p^2 \end{bmatrix}, \quad (30)$$

$$\mathbf{B} = \begin{bmatrix} \sum_{p=1}^{N_p} x_p (u_p - d) \\ \sum_{p=1}^{N_p} y_p (u_p - d) \\ \sum_{p=1}^{N_p} z_p (u_p - d) \end{bmatrix}. \quad (31)$$

Substituting the matrix $\mathbf{M}$ and vector $\mathbf{B}$ into equation (29) with their respective expressions from Equations (30) and (31), it is possible to rewrite an expression for the displacement gradient as:

$$\mathbf{G} = \sum_{p=1}^{N_p} \mathbf{g}_p u_p, \quad (32)$$

where $\mathbf{g}_p$ represents the least squares vector, defined for each vertex as:

$$\mathbf{g}_p = \begin{bmatrix} x_p^2 & x_p y_p & x_p z_p \\ x_p y_p & y_p^2 & y_p z_p \\ x_p z_p & y_p z_p & z_p^2 \end{bmatrix}^{-1} \begin{bmatrix} x_p - \frac{x_p}{N_p} \\ y_p - \frac{y_p}{N_p} \\ z_p - \frac{z_p}{N_p} \end{bmatrix}. \quad (33)$$

In index notation, equation (32) can be rewritten as:

$$G_{kl} = \sum_{p=1}^{N_p} (g_{kl})_p u_p. \quad (34)$$

In this work, it is assumed that the displacement gradient remains constant across each primary cell (linear variation of displacement within each primary cell). Under this assumption, the displacement gradient calculated using the least squares method represents the displacement gradient at the dual faces within each primary cell. Consequently, revisiting equation (20), the derivative of the displacement gradient with respect to the displacement can be obtained from equation (34). This relationship is expressed as:

$$\left(\frac{\partial G_{kl}}{\partial u_j} \right)_f = \sum_{p=1}^{N_p} (g_k \delta_{lj})_p, \quad (35)$$

where δ_{lj} is Kronecker delta. The presented discretisation is stabilised by replacing the displacement gradient component in the local primary edge direction using a central-differencing technique based on the immediate neighbours across the face. Using this approach, the gradient at a dual face is:

$$G_{kl} = \alpha e_k \frac{u_l^{nei} - u_l^{own}}{|x_l^{nei} - x_l^{own}|} + [\delta_{kn} - \alpha e_k e_n] \sum_{i=1}^C g_n^i u_l^i, \quad (36)$$

where the edge unit direction vector is:

$$e_k = \frac{x_k^{nei} - x_k^{own}}{|x_k^{nei} - x_k^{own}|}. \quad (37)$$

The central vertex position vector is x_k^{own} and the neighbouring vertex (associated with the dual face f) position vector is x_k^{nei} . In equation (36), the first term on the right-hand side presents the component of the gradient in the direction e_k , and the second term on the right-hand side represents the remaining components of the gradient. The scalar $0 \leq \alpha \leq 1$ gives control over the amount of the term that is replaced; for example, $\alpha = 1$ completely replaces the component in the e direction, whereas $\alpha = 0$ does not replace it at all. In the current work, $\alpha = 1$ is employed. This approach avoids zero energy modes, e.g. hourglassing. Consequently, the third order gradient tangent $\left(\frac{\partial G_{kl}}{\partial u_i} \right)_f$ then becomes:

$$\left(\frac{\partial G_{kl}}{\partial u_j} \right)_f = \alpha e_k \frac{\delta_{lj}^*}{|x_l^{nei} - x_l^{own}|} + [\delta_{kn} - \alpha e_k e_n] \sum_{i=1}^C g_n^i \delta_{lj}^i, \quad (38)$$

where $\delta_{lj}^* = \delta_{lj}$ for the neighbour vertex and $\delta_{lj}^* = -\delta_{lj}$ for the owner vertex.

3.2.4. Methodology for stress calculation at the primary vertices

This section elaborates on how the stress field can be calculated during post-processing. Two methodologies are employed in this work. The first approach involves calculating primary *cell* stresses directly using equation (4), where the constant displacement gradient within each primary cell is given by the least squares method (as detailed in Section 3.2.3). However, as shown in greater detail within the method of manufactured solutions in Section 4.2, *cell* stresses typically exhibit a first-order convergence rate. The reduced convergence order for stresses, as highlighted by Bathe [62], arises from the differentiation process intrinsic to stress computations. Specifically, stress derivation relies on the first derivative of the displacement field. When a function undergoes differentiation, its accuracy typically decreases by one order. Consequently, if the displacement field is approximated with an accuracy q , its derivative and, therefore, the resulting stress is approximated with a reduced accuracy of $q - 1$. To address this limitation, a second stress calculation approach is explored. The convergence order in vertex-centred methodologies can be improved by directly computing stresses at the primary vertices, as demonstrated by Vaz et al. [63]. In this approach, *point* stresses are computed using a least squares procedure applied to a compact computational stencil. This involves fitting a plane through the displacement values at neighbouring vertices that share an edge with the vertex P. By leveraging this localised fitting procedure, this method achieves a closer alignment with second-order accuracy on regular meshes. This methodology is similar to the stress recovery technique proposed by Zienkiewicz and Taylor [64].

3.2.5. Discretisation of the body forces

The body force contributions acting on each dual cell, as featured in equation (1), can be expressed in a discretised form as:

$$\int_{\Omega_p} \mathbf{F}_b d\Omega_p \approx \mathbf{F}_b \Omega_p, \quad (39)$$

where Ω_p represents the volume of the dual cell associated with primary vertex, P. This approximation is valid because the body force vector $\mathbf{F}_b$ is assumed to be constant across the dual cell. Furthermore, this method attains second-order accuracy, as each vertex serves as an approximate representation of the centroid of its corresponding dual cell, provided the cells are reasonably well-shaped and not excessively skewed. This approach leverages a central averaging technique, where the error resulting from assuming a constant force at the centroid scales with the square of the dual cell size.

3.3. Solution procedure

The final discretised form of the governing equation (equation (20)) for each primary vertex, P, can be rearranged as an algebraic equation:

$$\mathbf{A}_p \delta \mathbf{u}_p + \sum_{n=1}^{N_{ext}} \mathbf{A}_n \delta \mathbf{u}_n = \mathbf{b}_p. \quad (40)$$

where $\mathbf{A}_p$ and $\mathbf{A}_N$ are 3×3 tensors, $\mathbf{b}$ refers to the source vector and N_{ext} represents the set of all vertices within the extended stencil. This computational stencil for each vertex encompasses not only the neighbouring vertices that share an edge with vertex P , but also the additional vertices that form the cell to which the vertex belongs.

The algebraic equation defined in equation (40) is then assembled for all N_{tot} primary vertices into the following system:

$$[\mathcal{J}] [\delta \mathbf{u}] = [\mathbf{b}], \quad (41)$$

where the $3N_{\text{tot}} \times 3N_{\text{tot}}$ Jacobian matrix $[\mathcal{J}]$ is sparse with block-diagonal coefficients $\mathbf{A}_p$ and off-diagonal coefficients $\mathbf{A}_N$. The vector $[\delta \mathbf{u}]$ represents the unknown coupled displacement correction, structured for each computational node as:

$$[\delta \mathbf{u}] = \begin{bmatrix} \delta u \\ \delta v \\ \delta w \end{bmatrix}, \quad (42)$$

while $[\mathbf{b}]$ represents the source term, containing the residuals of the governing equation:

$$[\mathbf{b}] = \begin{bmatrix} -\text{Res}(u) \\ -\text{Res}(v) \\ -\text{Res}(w) \end{bmatrix}, \quad (43)$$

where $\text{Res}(u)$, $\text{Res}(v)$ and $\text{Res}(w)$ represent the components of the residual vector. The $[\delta \mathbf{u}]$ and $[\mathbf{b}]$ vectors for the entire domain each contain a total of $3N_{\text{tot}}$ entries.

The matrix system is solved for the displacement corrections $\delta \mathbf{u}$ in a fully coupled manner; that is, the unknown displacement components are solved simultaneously. Algorithm 3 shows the full block-coupled solution procedure. The outer iterations are performed using the Newton-Raphson algorithm, where the Jacobian matrix is updated once every outer iteration. The linear system formed at each outer iteration is then solved using the conjugate gradient iterative solver with ILU(k) preconditioner provided by PETSc [65], where k indicates the level of *fill-in* in the preconditioner. Finally, the total displacement and stresses are updated using the latest available values.

Algorithm 3 Displacement-based block-coupled solution procedure.

```

1: for all time steps do
2:   for all dual faces do
3:     Calculate the displacement gradient  $G$  (equation (34)).
4:     Calculate the stress  $\sigma$  (Algorithm 1).
5:     Calculate the material tangent  $C$  (Algorithm 2).
6:   end for
7:   while the Newton-Raphson residuals are not converged do
8:     Calculate coefficients of  $\mathcal{J}$  (equation (20)).
9:     Assemble the Jacobian matrix  $\mathcal{J}$  by adding the contributions from all dual faces.
10:    Calculate the source vector  $\mathbf{b}$  (equation (43)).
11:    Enforce the boundary conditions on the system (section 3.4).
12:    Solve the linear system  $[\mathcal{J}] [\delta \mathbf{u}] = [\mathbf{b}]$  using PETSc.
13:    Update the total displacement  $\mathbf{u}^{t+1} = \mathbf{u}^t + \delta \mathbf{u}$ .
14:    Update the displacement gradient at the dual faces (equation (34)).
15:    Update the stress at the dual faces (Algorithm 1).
16:   end while
17: end for

```

3.4. Implementation of the boundary conditions

Three types of boundary conditions are mainly used in this work: prescribed displacement (Dirichlet), prescribed traction (Neumann) and symmetry boundary conditions.

For *displacement* boundary conditions, the displacement is prescribed at the vertices on the boundaries. In the discretised equations for these vertices, the displacement correction $\delta \mathbf{u}$ is set to zero because the displacement is already prescribed and does not need to be updated. This significantly simplifies the discretised form of the linear momentum equation for these vertices as:

$$S_f \delta \mathbf{u}_p = 0, \quad (44)$$

where S_f represents a scaling factor introduced to preserve the conditioning of the linear system. In this work, the scaling factor is computed at the beginning of the simulation under the assumption of no plastic deformation, as follows:

$$S_f = (2\mu + \lambda) \sqrt{\frac{1}{N_d} \sum_{f=1}^{N_d} |\Gamma_f|}, \quad (45)$$

Table 1
Patch test. Mechanical properties.

Parameter	Value
Density, ρ	7,500 kg/m ³
Young's modulus, E	200 GPa
Poisson's ratio, ν	0.3

where N_d is the total number of dual faces in the domain. Conversely, in the equations for the internal vertices, the general form of the discretised equation is preserved, except that the displacement coefficients corresponding to neighbouring vertices with prescribed displacement are set to zero.

For *traction* boundary conditions, prescribed traction forces are applied to the boundary dual faces [66]. In this work, traction boundary conditions are enforced within the matrix system by incorporating the prescribed tractions into the residual of the momentum equation. This allows the residual vector to be rewritten as:

$$Res(u) = \sum_{f=1}^{N_f-N_b} \Gamma_f \cdot \sigma_f + \sum_{b=1}^{N_b} t_r |\Gamma_b|, \quad (46)$$

where t_r is the prescribed traction, N_b represents the number of dual faces located on the boundary where traction is applied and N_f is the total number of dual faces surrounding the boundary vertex. Consequently, the Jacobian matrix coefficients for these vertices are computed based solely on the internal dual faces, excluding the boundary dual faces, as neither the traction nor the surface area of the boundary dual faces depends on the displacement. This implies that the contributions from the traction boundaries are incorporated exclusively into the source term.

Finally, for *symmetry* boundary conditions, the displacement normal to a boundary vertex is constrained to zero, and the shear traction on the boundary dual faces is also set to zero [67]. These conditions are enforced using the same methodologies described for displacement and traction boundary conditions.

4. Benchmark cases

The performance of the developed vertex-centred block-coupled solver is verified on four distinct test cases:

- *Patch test*. This test case, initially proposed by Bazeley et al. [68] and analysed by Belytschko et al. [69], assesses the discretisation of the coupled solver for the ability to replicate first-order polynomials accurately. This assessment is carried out under elastic conditions to ascertain whether it yields zero discretisation errors when the displacement solution exhibits a linear variation across the domain.
- *Method of manufactured solutions*. This test case uses the method of manufactured solutions on a cubic domain to evaluate the order of accuracy of displacement and stress. This involves a comparison between the predicted and manufactured solutions.
- *Perforated plate and narrow T-member*. These test cases assess the ability of the coupled solver to solve complex geometry problems under small-strain elastic and elastoplastic conditions. Additionally, they are compared to vertex-centred and cell-centred segregated solvers implemented in *solids4foam* by Cardiff et al. [70,71], in terms of accuracy and computational efficiency.

An additional clarification regarding the notation used in the next sections is needed. The abbreviations are as follows: “VC” stands for vertex-centred and “CC” for cell-centred.

4.1. Patch test

The *patch test*, initially introduced by Bazeley et al. [68] and analysed by Belytschko et al. [69], evaluates the solver's discretisation to accurately model first-order polynomials, specifically the displacement field. This property, known as *polynomial completeness*, requires the discretisation to produce zero errors when the true displacement solution is linear.

The geometry consists of an irregular quadrilateral with the following dimensions: $a = 0.35$ m, $b = 0.36$ m, $c = 0.26$ m, $d = 0.30$ m, and with an angle $\delta = 75^\circ$. The domain is additionally divided into irregular quadrilateral cells, as shown in Fig. 2. Selecting distorted cells is crucial for this test case, as orthogonal cells might pass the patch test, while non-orthogonal cells may not. The four boundaries are subject to a linearly varying displacement. The x - and y -components of the displacement on the boundaries, u and v , are defined by equations (47) and (48):

$$u = \alpha_{x_0} + \alpha_{x_1} x + \alpha_{x_2} y, \quad (47)$$

$$v = \alpha_{y_0} + \alpha_{y_1} x + \alpha_{y_2} y, \quad (48)$$

where x and y refer to the Cartesian coordinates of the boundary vertices, and the α parameters are constants set by the user. In this test case, $\alpha_{x_0} = 1 \times 10^{-6}$, $\alpha_{x_1} = 2 \times 10^{-6}$, $\alpha_{x_2} = 5 \times 10^{-6}$, $\alpha_{y_0} = 4 \times 10^{-6}$, $\alpha_{y_1} = 3 \times 10^{-6}$ and $\alpha_{y_2} = 6 \times 10^{-6}$. Additionally, the mechanical properties are outlined in Table 1.

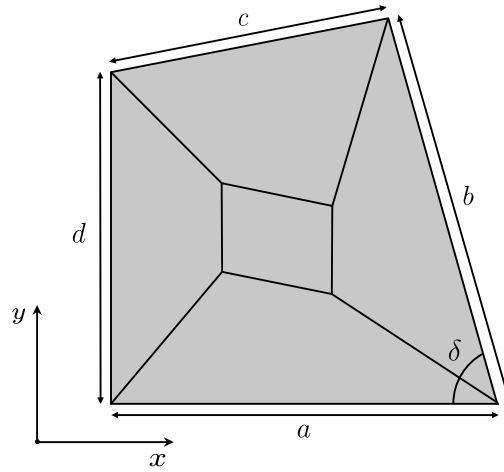


Fig. 2. Patch test. Geometry divided into five irregular quadrilateral cells.

Table 2

Patch test. Predicted and analytical solutions of strains and stresses, rounded up to 10 decimal places.

Variables	Predicted	Analytical	% error
ϵ_{xx}	$1.9999999949 \times 10^{-6}$	$2.0000000000 \times 10^{-6}$	3×10^{-7}
ϵ_{yy}	$6.0000002122 \times 10^{-6}$	$6.0000000000 \times 10^{-6}$	4×10^{-6}
ϵ_{xy}	$3.9999999899 \times 10^{-6}$	$4.0000000000 \times 10^{-6}$	3×10^{-7}
σ_{xx} (Pa)	1.2307700000×10^6	1.2307692307×10^6	6×10^{-5}
σ_{yy} (Pa)	1.8461500000×10^6	1.8461538461×10^6	2×10^{-4}
σ_{xy} (Pa)	0.6153850000×10^6	0.6153846153×10^6	6×10^{-5}

The proposed discretisation successfully passes the patch test since the strains and stresses are observed constant across the entire domain with predicted values shown in Table 2. The analytical values used for comparison with the predicted values are directly calculated from the displacement expressions in equations (47) and (48): The xx -, yy - and xy -components of the analytical strain are calculated as:

$$\epsilon_{xx} = \frac{\partial u}{\partial x} = \alpha_{x_1}, \quad (49)$$

$$\epsilon_{yy} = \frac{\partial v}{\partial y} = \alpha_{y_2}, \quad (50)$$

$$\epsilon_{xy} = \frac{1}{2} \left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \right) = \frac{1}{2} (\alpha_{x_2} + \alpha_{y_1}). \quad (51)$$

Similarly, the analytical stresses are calculated using equation (3):

$$\sigma_{xx} = 2\mu\epsilon_{xx} + \lambda(\epsilon_{xx} + \epsilon_{yy}), \quad (52)$$

$$\sigma_{yy} = 2\mu\epsilon_{yy} + \lambda(\epsilon_{xx} + \epsilon_{yy}), \quad (53)$$

$$\sigma_{xy} = 2\mu\epsilon_{xy}. \quad (54)$$

The predicted strains and stresses from the developed coupled solver are compared to the analytical solutions, with values rounded up to 10 decimal places. The percentage errors between the predicted values, ϕ_{pred} , and the analytical values, ϕ_{an} , are calculated as:

$$\% \text{ error} = \left| \frac{\phi_{\text{an}} - \phi_{\text{pred}}}{\phi_{\text{an}}} \right| \times 100\%. \quad (55)$$

The comparison in Table 2 demonstrates excellent agreement between predicted and analytical values, with negligible percentage errors ranging from $10^{-4}\%$ to $10^{-7}\%$. As a result, the discretisation used in the developed coupled solver successfully passes the patch test. The predicted strains and stresses remain constant throughout the domain, with minimal percentage errors, demonstrating the discretisation's ability to reproduce first-order polynomials accurately.

Table 3
Method of manufactured solutions. Number of primary cells and vertices.

Mesh	Structured tetrahedra		Unstructured tetrahedra		Structured hexahedra	
	Cells	Vertices	Cells	Vertices	Cells	Vertices
Mesh 1	384	125	364	131	64	125
Mesh 2	3,072	729	2,580	716	512	729
Mesh 3	24,576	4,913	16,485	3,740	4,096	4,913
Mesh 4	196,608	35,937	125,363	24,152	32,768	35,937
Mesh 5	1,572,864	274,625	941,309	166,717	262,144	274,625

4.2. Method of manufactured solutions

The method of manufactured solutions is used to verify the order of convergence of numerical methods. In this approach, a manufactured solution is chosen, and the governing equations are modified by introducing source terms that ensure the selected solution satisfies them. These source terms are treated as known, allowing the numerical method to be tested against a solution with a known exact answer. The order of convergence of the numerical method can be evaluated by comparing the predicted results with the manufactured solution. In this work, the analytical solutions for the three components of displacement, u , v and w , are defined using trigonometric functions as follows:

$$u = 2 \sin(4\pi x) \sin(2\pi y) \sin(\pi z), \quad (56)$$

$$v = 4 \sin(4\pi x) \sin(2\pi y) \sin(\pi z), \quad (57)$$

$$w = 6 \sin(4\pi x) \sin(2\pi y) \sin(\pi z). \quad (58)$$

The geometry consists of a cube with a side length $L = 1$ m. In this test case, Dirichlet boundary conditions are imposed on all sides of the cube, with values set to zero since the manufactured solutions are exactly zero at the boundaries. This test case employs the same mechanical properties as the patch test (Table 1).

Five structured and unstructured tetrahedral meshes, as well as structured hexahedral meshes, generated using *Gmsh* [72], with the number of primary cells and vertices summarised in Table 3, are analysed to compare the order of accuracy of the three mesh types and to evaluate further the robustness of the developed coupled solver. Fig. 3 presents a cross-sectional view of the three mesh configurations corresponding to mesh 3.

In Fig. 4 the distributions of displacement magnitude along the y - z plane at position $x = 0.85$ m for the finest meshes (mesh 5) are illustrated. The predicted solutions are compared against the corresponding manufactured solution with the displacement magnitude, $|\mathbf{u}|$, is computed as:

$$|\mathbf{u}| = \sqrt{u^2 + v^2 + w^2}. \quad (59)$$

The comparison between the predicted and manufactured solutions reveals a strong agreement for both tetrahedral and hexahedral meshes, verifying the robustness of the developed methodology across different mesh types.

The order of convergence of the solutions, specifically regarding displacement magnitude and von Mises stress, is further examined through the calculation of three distinct mathematical error norms. The L_1 norm represents the average absolute error and can be expressed as:

$$L_1 = \frac{1}{N_{\text{tot}}} \sum_{i=1}^{N_{\text{tot}}} |\Delta\phi_i|, \quad (60)$$

where $\Delta\phi_i$ is the difference between the manufactured and predicted solutions and N_{tot} is the number of primary vertices.

The L_2 norm represents the root mean square error and is expressed as:

$$L_2 = \sqrt{\frac{1}{N_{\text{tot}}} \sum_{i=1}^{N_{\text{tot}}} \Delta\phi_i^2}. \quad (61)$$

Finally, the L_∞ norm represents the maximum absolute error:

$$L_\infty = \max_{1 \leq i \leq N_{\text{tot}}} |\Delta\phi_i|. \quad (62)$$

The error norms are computed for the tetrahedral and hexahedral meshes listed in Table 3, allowing for the analysis of the convergence order for displacement magnitude and von Mises stress. The latter is calculated as:

$$\sigma_v = \sqrt{\frac{1}{2} \left[(\sigma_{xx} - \sigma_{yy})^2 + (\sigma_{yy} - \sigma_{zz})^2 + (\sigma_{zz} - \sigma_{xx})^2 \right] + 3(\sigma_{xy}^2 + \sigma_{yz}^2 + \sigma_{zx}^2)}. \quad (63)$$

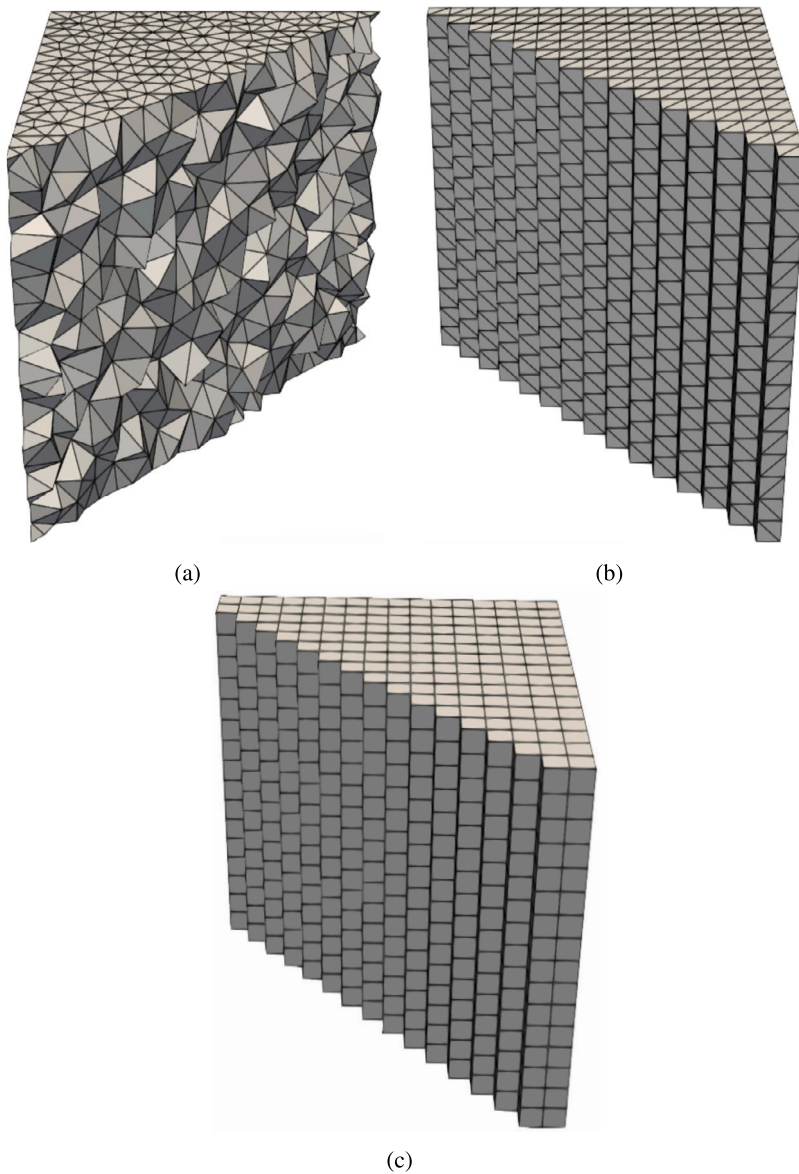


Fig. 3. Method of manufactured solutions. Cross-sectional view showing (a) an unstructured tetrahedral mesh, containing 16,485 cells and 3,740 vertices, (b) a structured tetrahedral mesh, comprising 24,576 cells and 4,913 vertices and (c) a structured hexahedral mesh with 4,096 cells and 4,913 vertices.

Fig. 5 illustrates the average absolute error, L_1 , as a function of the average mesh spacing for both tetrahedral and hexahedral meshes. The convergence behaviour of the developed coupled solver is then compared against the CC segregated solver implemented in *solids4foam* by Cardiff et al. [70,71] for reference. These plots display an order of convergence lower than second-order for coarse meshes, with the convergence rate improving to approximately second-order as the mesh is refined, as expected. A comparison with the CC segregated approach reveals similar convergence rates. However, as shown in Fig. 5b, the *point* von Mises stress in the unstructured tetrahedral mesh achieves a lower convergence rate than the corresponding *point* von Mises stress in the structured mesh. This discrepancy indicates that the irregularity of the unstructured mesh introduces greater sensitivity to mesh quality. Furthermore, the proposed discretisation consistently yields lower average absolute errors for displacement and von Mises stress when compared to the CC segregated solver. This indicates a closer alignment of the coupled solver's results with the manufactured solution.

The order of convergence of the L_1 , L_2 and L_∞ norms are summarised in Tables 4, 5 and 6. These values are calculated using the finest two meshes (i.e., mesh 4 and mesh 5), demonstrating the order of convergence achieved by the coupled solver for the three mesh types. The convergence behaviour of the displacement magnitude exhibits an approximately second-order convergence rate across all error norms as the mesh is refined. Furthermore, the *point* von Mises stress shows improved convergence for the L_1 and L_2 norms, attaining second-order accuracy compared to the *cell* von Mises stress. The lower convergence order for the L_∞ norm may be attributed to the maximum stress error occurring at the boundaries. These convergence rates are consistent with those obtained by the

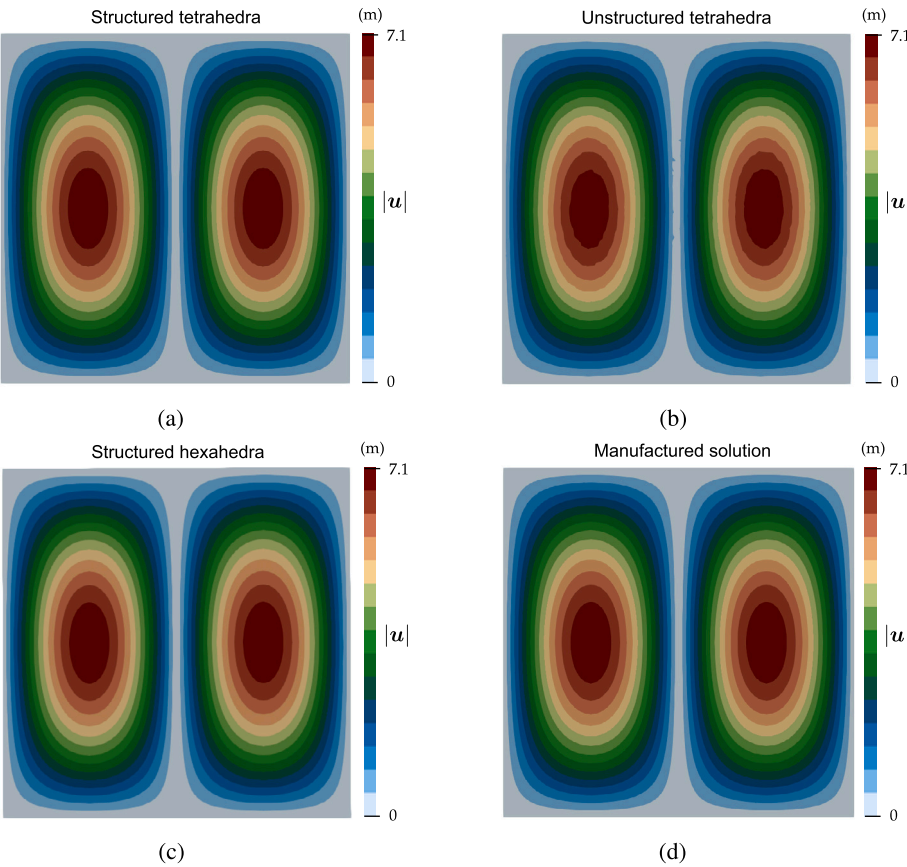


Fig. 4. Method of manufactured solutions. Distribution of the displacement magnitude along the y - z plane at position $x = 0.85$ m. A comparison between the numerical solutions provided by the developed coupled solver for (a) the structured tetrahedral mesh, (b) the unstructured tetrahedral mesh, (c) the structured hexahedral mesh and (d) the manufactured solution is shown. The presented solutions pertain to mesh 5.

Table 4
Method of manufactured solutions. Order of convergence of displacement magnitude and von Mises stress for the structured tetrahedral mesh.

Norm	Coupled (VC)			Segregated (CC)	
	Displacement	Stress (<i>cell</i>)	Stress (<i>point</i>)	Displacement	Stress
L_1	1.88068	1.01310	1.83537	1.82868	1.77136
L_2	1.93218	1.01086	1.68675	1.84921	1.71155
L_∞	1.93306	0.97694	0.98260	1.90263	1.20942

Table 5
Method of manufactured solutions. Order of convergence of displacement magnitude and von Mises stress for the unstructured tetrahedral mesh.

Norm	Coupled (VC)			Segregated (CC)	
	Displacement	Stress (<i>cell</i>)	Stress (<i>point</i>)	Displacement	Stress
L_1	1.85566	1.04693	1.52473	1.96489	1.79100
L_2	1.96303	1.05305	1.50094	1.95406	1.73181
L_∞	1.77418	0.89329	1.28764	1.84634	1.18266

CC segregated solver, except for the *point* von Mises stress in the unstructured tetrahedral mesh, which shows lower L_1 and L_2 norms compared to the CC segregated solver. Overall, the analysis confirmed that the developed VC coupled solver meets the theoretical convergence expectations, with structured tetrahedral and hexahedral meshes performing slightly better in terms of convergence rates.

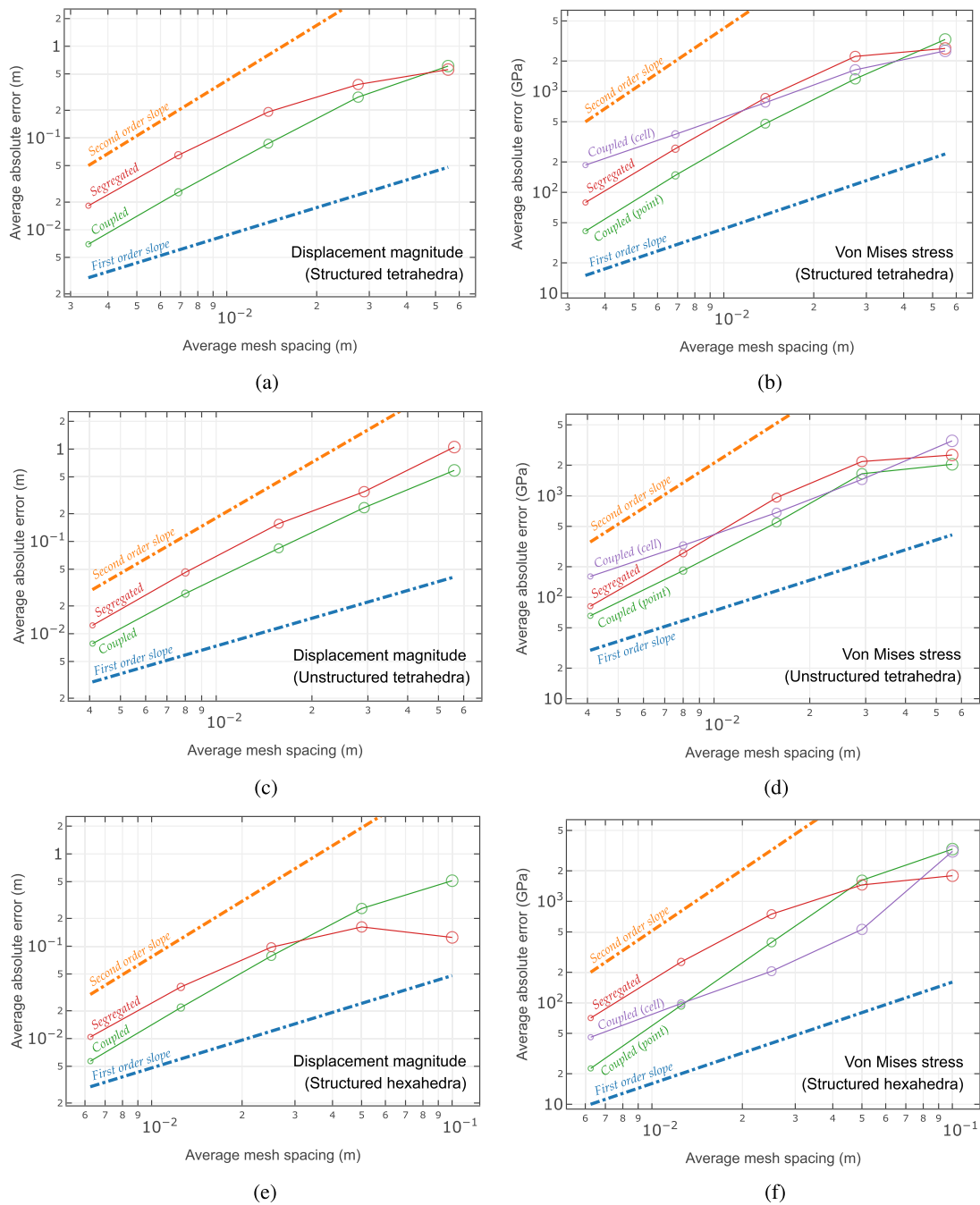


Fig. 5. Method of manufactured solutions. Average absolute error as a function of the average mesh spacing. (a), (c) and (e) refer to the displacement magnitude, and (b), (d) and (f) refer to the cell and point von Mises stresses for structured tetrahedral, unstructured tetrahedral and structured hexahedral mesh types, respectively.

Table 6
Method of manufactured solutions. Order of convergence of displacement magnitude and von Mises stress for the structured hexahedral mesh.

Norm	Coupled (VC)			Segregated (CC)	
	Displacement	Stress (cell)	Stress (point)	Displacement	Stress
L_1	1.92877	1.10458	2.05913	1.79911	1.83529
L_2	1.97207	1.19994	1.92489	1.84968	1.84076
L_∞	1.98201	0.90464	0.98268	1.87246	0.93753

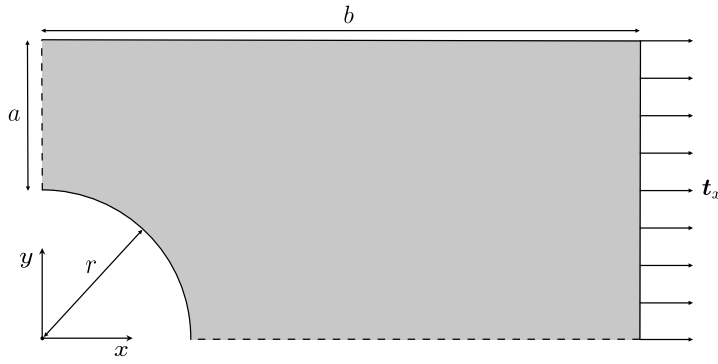


Fig. 6. Perforated plate. Geometry and loading. The dotted lines represent symmetry axes.

Table 7
Perforated plate. Number of primary cells and vertices.

Mesh	Structured triangles		Unstructured triangles	
	Cells	Vertices	Cells	Vertices
Mesh 1	550	632	470	530
Mesh 2	2,200	2,362	1,896	2,014
Mesh 3	8,800	9,122	7,666	7,900
Mesh 4	32,200	35,842	30,465	30,930

Table 8
Perforated plate. Mechanical properties.

Parameter	Value
Density, ρ	7,500 kg/m ³
Young's modulus, E	70 GPa
Poisson's ratio, ν	0.3
Yield stress, σ_y	243 MPa

4.3. Perforated plate

A 2-D plate is perforated by a circular hole with radius $r = 0.5$ m and subjected to time-varying uniaxial traction, t_x , over the duration of 20 seconds [70]. This traction is expected to induce an increasing amount of plastic deformation propagating across the domain, which culminates at 10 seconds with the applied maximum traction of 133.65 MPa. Subsequently, the traction undergoes unloading, accompanied by a sudden decrease in plasticity. The following equation describes the applied traction over time:

$$t_x = \begin{cases} \frac{133.65 \text{ MPa}}{10} t, & 0 \leq t \leq 10 \text{ s} \\ 133.65 \text{ MPa} \left(1 - \frac{t-10}{10}\right), & 10 \text{ s} < t \leq 20 \text{ s} \end{cases} \quad (64)$$

Due to the symmetry of the plate geometry along both Cartesian axes, only one-quarter of the structure, with dimensions $a = 0.5$ m and $b = 2$ m, is modelled, as shown in Fig. 6.

Four structured and unstructured triangular meshes, generated using *Gmsh* [72], with the number of primary cells and vertices summarised in Table 7, are analysed using the developed coupled solver. Fig. 7 shows the structured and unstructured configurations corresponding to mesh 2. Additionally, this test case is modelled under the assumption of perfect plasticity with mechanical properties summarised in Table 8.

The initial analysis of this test case involves monitoring the stress distribution across the plate over multiple time steps. Fig. 8 shows the xx -component of stress, σ_{xx} , at two time steps: at 10 seconds, representing the moment of maximum loading applied to the plate and at 20 seconds, when the traction is completely unloaded. Fig. 8a illustrates that σ_{xx} reaches its peak when the plate is subjected to maximum loading at 10 seconds. The highest σ_{xx} , measuring 360 MPa, is observed above the hole where the plate experiences plastic deformation. Moving forward to Fig. 8b, the traction decreases to zero at 20 seconds, causing a significant reduction in stress across most of the plate. However, σ_{xx} goes into compression (-93 MPa) along the left rim of the plate above the hole. A direct evaluation is conducted to compare the results from the developed VC coupled solver with those from the CC segregated solver, demonstrating strong agreement.

In Fig. 9a the profile of σ_{xx} along $x = 0$ is depicted at 10 seconds. At this time step, the region surrounding the hole exhibits the maximum plastic deformation, as evidenced by a notable increase in stress near the hole. This stress reaches its pinnacle at $y = 0.68$ m,

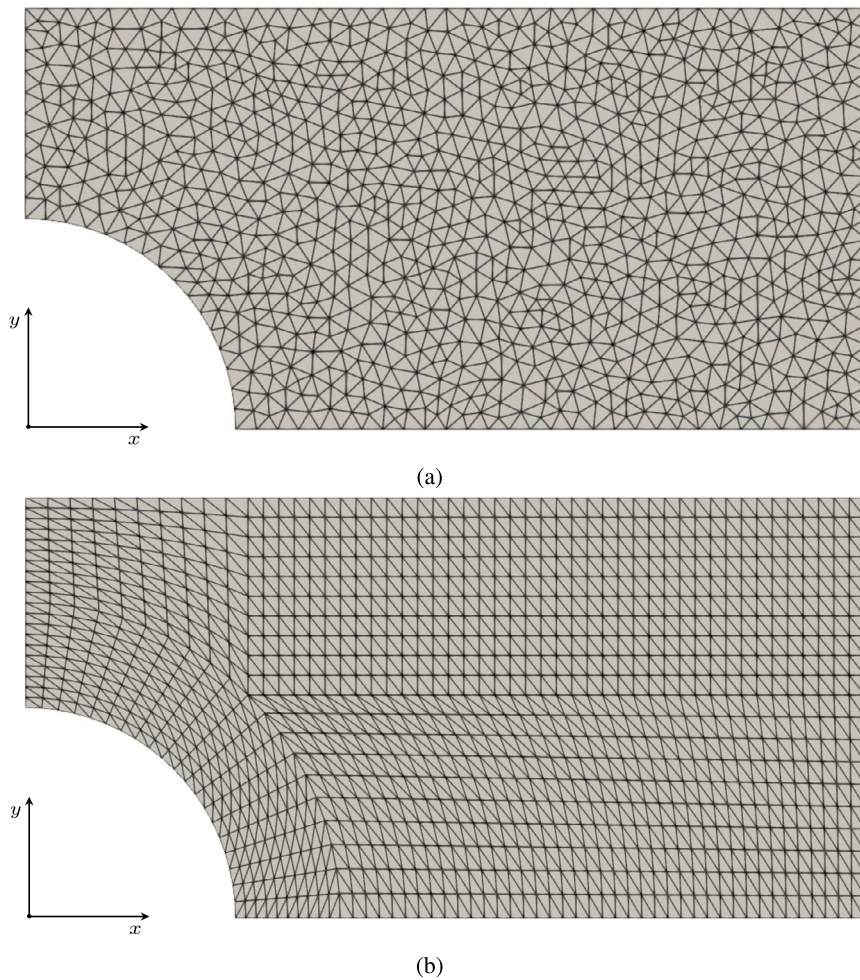


Fig. 7. Perforated plate. (a) Unstructured triangular mesh containing 1,896 cells and 2,014 vertices, and (b) structured triangular mesh containing 2,200 cells and 2,362 vertices.

attaining a value of 365 MPa, and gradually diminishes with increasing distance from the hole. In Fig. 9b, corresponding to 20 seconds, the plate experiences no plastic deformation as the traction returns to zero. Notably, stress patterns indicate compressive forces near the hole and expansive forces toward the upper side of the plate. A stress peak of 96 MPa is registered at $y = 0.73$ m, followed by a gradual decrease in stress that transitions back to compressive stresses at the upper portion of the plate. The analysis extends in Fig. 9c to the evolution of the v -displacement at the uppermost point of the hole within the time range of 1 to 20 seconds. The observed profile reveals a distinct V-shaped pattern reaching a minimum value of -15×10^{-4} m at 10 seconds. The presented solutions are derived from four progressively refined structured meshes, showcasing that as the number of cells increases, both the stress and the displacement align closely with those generated by the CC segregated solver.

The efficiency of the methodology is assessed in terms of time and memory requirements. The VC coupled solver is run using a conjugate gradient iterative solver with ILU preconditioning with the displacement tolerance of the outer Newton-Raphson iterations set at 10^{-11} . The performance of the coupled solver is also analysed against a VC and CC segregated solvers, which are executed using a conjugate gradient iterative solver with ILU preconditioning, with a displacement tolerance set at 10^{-6} . The simulations are run using 1 core of an Intel i7-10750H CPU at 2.60 GHz over 100 quasi-static time steps, each equal to 0.2 s.

Tables 9 and 10 show a comparison between the VC coupled solver and the VC and CC segregated solvers analysed in terms of computational efficiency for both structured and unstructured triangular meshes, revealing that the VC coupled solver requires less time than the other solvers, albeit at the cost of increased memory usage, as expected.

In Table 11 the effective speedups for the VC coupled solver relative to the other solvers are shown. An important consideration arises when comparing the VC coupled solvers to the CC segregated solver. In OpenFOAM, even for a 2-D test case, the domain is treated as a 3-D geometry, just one cell thick. This setup results in two layers of vertices: the front and back faces of the domain. Consequently, in the current implementation, the VC solver solves the governing equations at the nodes on both layers, whereas the CC solver solves the equations for only a single layer of cell centres. This implementation difference should be taken into account

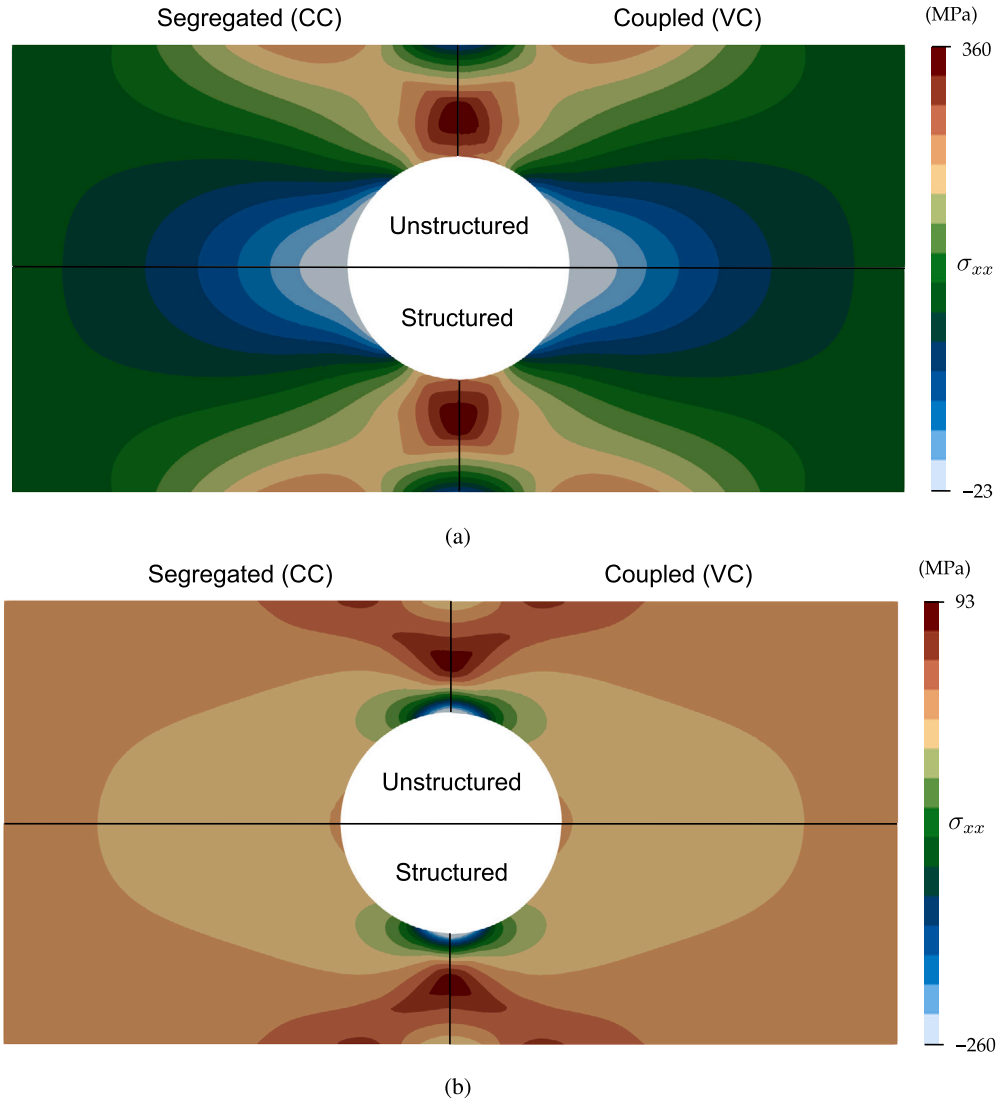


Fig. 8. Perforated plate. Distribution of σ_{xx} at (a) 10 seconds and (b) 20 seconds. A direct comparison between the results provided by the developed VC coupled solver and a CC segregated solver is shown for both structured and unstructured triangular meshes. The results correspond to the finest mesh analysed (mesh 4).

Table 9

Perforated plate. Wall-clock times and maximum memory usage for the structured triangular meshes.

Mesh	Coupled (VC)		Segregated (VC)		Segregated (CC)	
	Time (s)	RAM (MB)	Time (s)	RAM (MB)	Time (s)	RAM (MB)
Mesh 1	10	19	49	11	21	8
Mesh 2	40	65	229	17	67	13
Mesh 3	198	247	1,127	39	345	33
Mesh 4	1,023	952	5,689	127	1,894	124

when comparing the two approaches for the 2-D cases. Therefore, for 2-D geometries, the following equation is used to calculate the effective speedups of the VC coupled solver compared to the CC segregated solver:

$$\text{speedup} = 2 \left(\frac{t_{cc}}{t_{vc}} \right) \left(\frac{\text{vertices}}{\text{cells}} \right), \quad (65)$$

where t_{cc} and t_{vc} are the wall-clock times recorded by the CC and VC solvers, respectively, and “vertices” and “cells” refer to the total number of vertices and cells in the domain. Conversely, the speedups of the VC coupled solver relative to the VC segregated solver

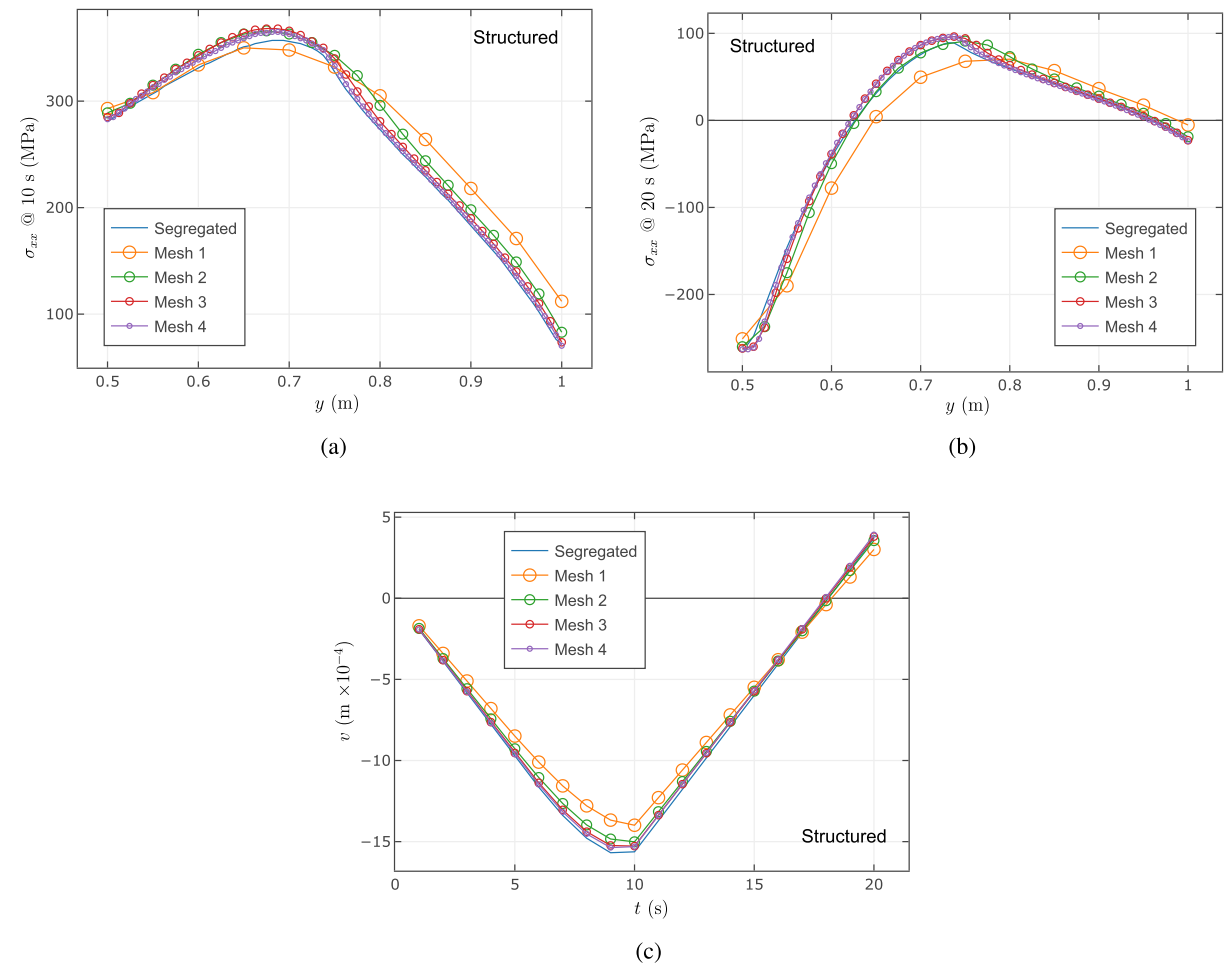


Fig. 9. Perforated plate. σ_{xx} profile along $x = 0$ at (a) 10 seconds and (b) 20 seconds. (c) v -displacement captured at the uppermost point of the hole with coordinates (0,0.5) for multiple time steps. These plots are obtained using four structured triangular meshes of an increasing number of cells and compared to the solution given by the CC segregated solver on mesh 4.

Table 10
Perforated plate. Wall-clock times and maximum memory usage for the unstructured triangular meshes.

Mesh	Coupled (VC)		Segregated (VC)		Segregated (CC)	
	Time (s)	RAM (MB)	Time (s)	RAM (MB)	Time (s)	RAM (MB)
Mesh 1	6	17	43	7	16	6
Mesh 2	27	59	156	13	47	10
Mesh 3	151	201	978	32	230	26
Mesh 4	938	739	4,271	119	1,249	97

are determined by directly dividing their respective wall-clock times. Overall, the developed VC coupled solver achieves speedups ranging from 4.6× and 6.5× relative to the VC segregated solver, and from 2.7× and 6.0× in comparison to the CC segregated solver. It is noted that the effective speedup calculations presented in this work reflect practical usage within the current constraints of OpenFOAM. In particular, for 2-D test cases, OpenFOAM requires a 3-D mesh (one layer of cells), leading to a duplication of vertices in the third dimension. This duplication introduces a computational overhead for VC approaches that is not inherent to the method itself. As such, the reported speedup values may underestimate the ideal performance of the VC coupled solver in purely 2-D applications. While this limitation is outside user control in the current framework, future work will explore mesh utilities or pre-processing strategies to better support efficient 2-D simulations. The speedup metric reported here is based on execution time per degree of freedom and does not account for differences in numerical accuracy between methods. However, as shown in the manufactured solution test case (Section 4.2), the VC and CC

Table 11

Perforated plate. Effective speedups of the VC coupled solver compared to VC and CC segregated solvers.

Mesh	Segregated (VC)		Segregated (CC)	
	Structured	Unstructured	Structured	Unstructured
Mesh 1	4.9×	7.2×	4.8×	6.0×
Mesh 2	5.7×	5.7×	3.6×	3.7×
Mesh 3	5.7×	6.5×	3.6×	3.1×
Mesh 4	5.6×	4.6×	4.1×	2.7×

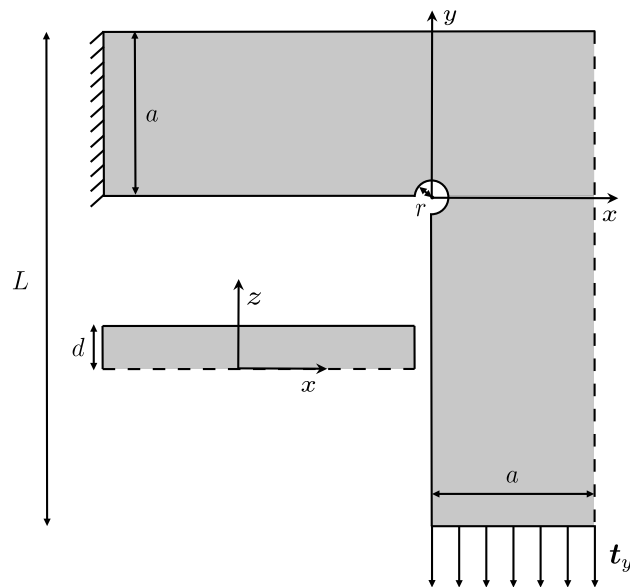


Fig. 10. Narrow T-member. Top and side views, subjected to a unidirectional load. The dotted lines represent symmetry planes.

approaches exhibit comparable accuracy for a given mesh. Therefore, execution time per degree of freedom provides a reasonable and informative basis for comparing solver efficiency in this context.

4.4. Narrow T-member

A 3-D narrow member featuring a T-cross section has been proposed by Demirdžić et al. [3] and analysed by Cardiff et al. [24] as a typical engineering component. To mitigate stress concentration, a hole with a radius $r = 0.005$ m is drilled at the angle between the flange and the web. Since the geometry is symmetric, only one-quarter is analysed herein with dimensions $L = 0.15$ m and $a = 0.05$ m, and thickness $d = 0.015$ m, shown in Fig. 10. Additionally, the left side of the T-member is fixed, while a constant downward-pointing traction $t_y = 1$ MPa is applied to the lower part of the web.

Six structured tetrahedral and hexahedral meshes, generated using *Gmsh* [72], with the number of primary cells and vertices summarised in Table 12, are analysed using the developed coupled solver. Figs. 11a and 11b show the tetrahedral and hexahedral meshes corresponding to mesh 1. While the original test case focused on elastic conditions, this work extends the problem to elastoplastic behaviour, where perfect plasticity is assumed, with the mechanical properties listed in Table 13. The yield stress, σ_y , is set to 3.8 MPa, whose value was selected to induce a controlled amount of plastic deformation, avoiding purely elastic behaviour or excessive plasticity. It results in plastic deformation affecting up to 31.25% of the cells in the domain.

The distribution of von Mises stress along the $z = 0.015$ m plane, illustrated in Fig. 12, provides insight into the specific locations where plastic deformation is taking place within the geometry under consideration. Plastic deformation occurs when the von Mises stress reaches the material's yield stress (i.e. 3.8 MPa). In the context of Fig. 12, these instances are visually represented by the regions in the vicinity of the hole and extending towards the far side of the flange. A comparative analysis of the results obtained on tetrahedral and hexahedral meshes reveals the von Mises stress predictions of the VC coupled solver and the CC segregated solver to agree in the limit of mesh refinement.

In Fig. 13a, the von Mises stress profile along the x -axis at $z = 0.015$ m is depicted for tetrahedral meshes of an increasing number of cells. Notably, the von Mises stress exhibits a rising trend towards the hole position, denoting the region of highest stress concentration. Furthermore, it plateaus at approximately 0.124 m featuring a maximum von Mises stress of 3.8 MPa, coinciding with the material's yield stress. This observation aligns with expectations, reflecting the material's shift from elastic to plastic deformation at this specific location. Fig. 13b shows the von Mises stress profile along the same line for hexahedral meshes, indicating that the VC

Table 12
Narrow T-member. Number of primary cells and vertices.

Mesh	Tetrahedra		Hexahedra	
	Cells	Vertices	Cells	Vertices
Mesh 1	5,880	1,662	980	1,662
Mesh 2	47,040	10,435	7,840	10,435
Mesh 3	158,760	32,200	26,460	32,200
Mesh 4	376,320	72,837	62,720	72,837
Mesh 5	750,000	138,226	122,500	138,226
Mesh 6	1,270,080	234,247	211,680	234,247

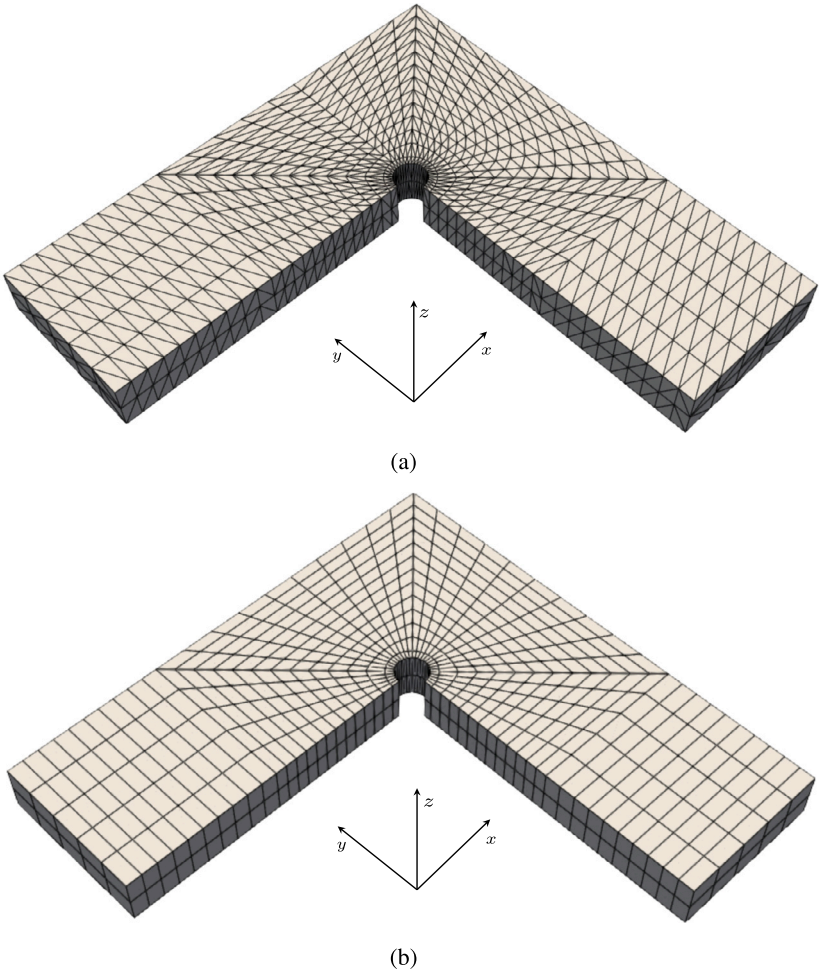


Fig. 11. Narrow T-member. (a) Structured tetrahedral mesh containing 5,880 cells and 1,662 vertices, and (b) structured hexahedral mesh containing 980 cells and 1,662 vertices.

Table 13
Narrow T-member. Mechanical properties.

Parameter	Value
Density, ρ	7,500 kg/m ³
Young's modulus, E	210 GPa
Poisson's ratio, ν	0.3
Yield stress, σ_y	3.8 MPa

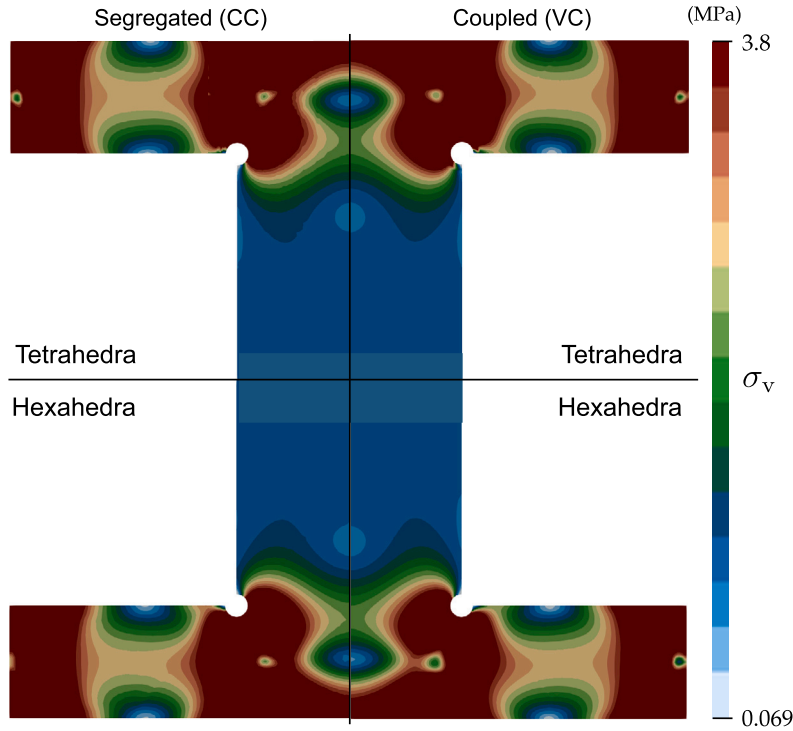


Fig. 12. Narrow T-member. Distribution of von Mises stress, showing a comparison between the results provided by the developed VC coupled solver and the CC segregated solver on tetrahedral and hexahedral meshes. The results correspond to the finest mesh analysed (mesh 6).

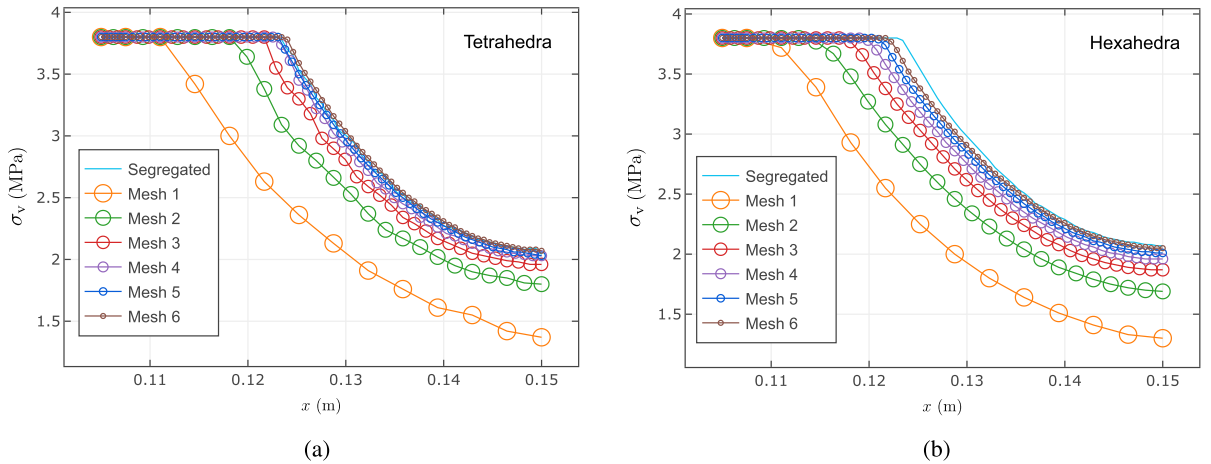


Fig. 13. Narrow T-member. Von Mises stress profile along the x -axis at $z = 0.015$ m. The results are obtained from (a) structured tetrahedral meshes and (b) structured hexahedral meshes of an increasing number of cells and are compared to the CC segregated solution on mesh 6.

coupled solver slightly underpredicts the onset of plastic deformation for the same mesh density, as evidenced by the small disparity in von Mises stress compared to the CC segregated solver.

The efficiency of the methodology is assessed in terms of time and memory requirements, consistent with the test case in Section 4.3. The coupled solver is run using the conjugate gradient iterative solver with ILU preconditioning with the displacement tolerance of the outer Newton-Raphson iterations set at 10^{-11} . The performance of the VC coupled solver is also analysed against VC and CC segregated solvers, which are executed using a conjugate gradient iterative solver with ILU preconditioning, with a displacement tolerance set at 10^{-6} . The simulations are run using 1 core of an Intel i9-10900X CPU at 3.70 GHz over one quasi-static time step of 1 s.

Tables 14 and 15 provide a comparative analysis of wall-clock times and memory usage for the developed VC coupled solver, alongside the VC and CC segregated solvers, for both tetrahedral and hexahedral meshes. The analysis reveals that the developed VC coupled solver achieves a significant reduction in wall-clock times compared to the other solvers, despite the higher memory con-

Table 14

Narrow T-member. Wall-clock times and maximum memory usage for the tetrahedral meshes.

Mesh	Coupled (VC)		Segregated (VC)		Segregated (CC)	
	Time (s)	RAM (MB)	Time (s)	RAM (MB)	Time (s)	RAM (MB)
Mesh 1	4	188	9	73	31	97
Mesh 2	36	1,145	278	189	1,096	305
Mesh 3	158	3,667	4,215	530	11,061	865
Mesh 4	1,106	8,037	31,831	1,223	47,986	1,950
Mesh 5	2,796	14,300	103,427	2,945	145,537	3,730
Mesh 6	8,914	23,200	207,618	4,132	398,455	5,805

Table 15

Narrow T-member. Wall-clock times and maximum memory usage for the hexahedral meshes.

Mesh	Coupled (VC)		Segregated (VC)		Segregated (CC)	
	Time (s)	RAM (MB)	Time (s)	RAM (MB)	Time (s)	RAM (MB)
Mesh 1	3	186	10	79	9	63
Mesh 2	44	1,124	302	198	165	179
Mesh 3	167	3,786	4,532	589	3,526	564
Mesh 4	1,182	8,236	35,629	1,202	28,031	1,146
Mesh 5	3,247	14,800	107,478	3,048	67,287	2,890
Mesh 6	10,517	22,700	215,390	4,245	175,836	4,029

Table 16

Narrow T-member. Effective speedups of the coupled solver compared to the VC and CC segregated solvers.

Mesh	Segregated (VC)		Segregated (CC)	
	Tetrahedra	Hexahedra	Tetrahedra	Hexahedra
Mesh 1	2.3×	3.3×	2.0×	5.1×
Mesh 2	7.7×	6.9×	6.8×	5.0×
Mesh 3	26.7×	27.1×	14.2×	25.7×
Mesh 4	28.8×	30.1×	8.4×	27.5×
Mesh 5	37.0×	33.1×	9.6×	23.4×
Mesh 6	23.3×	20.5×	8.2×	18.5×

sumption. Additionally, it was observed that the VC coupled solver tends to perform faster on tetrahedral meshes than on hexahedral meshes, particularly on finer meshes, even though both mesh types contain the same number of computational nodes. This suggests that tetrahedral meshes may be more efficient for this particular problem.

Table 16 presents a comparison of speedups for the VC coupled solver relative to the other solvers analysed. In particular, the speedup of the VC coupled solver compared to the CC segregated solver is calculated using the following equation for 3-D geometries:

$$\text{speedup} = \left(\frac{t_{\text{cc}}}{t_{\text{vc}}} \right) \left(\frac{\text{vertices}}{\text{cells}} \right). \quad (66)$$

Overall, the VC coupled solver achieves speedups ranging from 2.3×

 and 37.0× relative to the VC segregated solver and from 2.0× and 27.5× in comparison to the CC segregated solver.

Finally, the convergence behaviour of the Newton-Raphson iterations in the developed VC coupled solvers is analysed to verify it aligns with the theoretical expectations of the method. According to the Newton-Raphson algorithm, the residuals should decrease at a second-order rate as the current solution approaches the true solution. This analysis aims to ensure that the solver's convergence not only reaches the prescribed tolerance but also adheres to the expected quadratic convergence of the Newton-Raphson procedure as the solution approaches the true values.

Fig. 14 illustrates the convergence behaviour of the relative displacement residuals for both tetrahedral and hexahedral meshes. An analysis of Figs. 14a and 14b reveals that the relative residuals exhibit stagnation during the initial iterations, particularly for the finer mesh resolutions. This plateau phase is followed by a rapid convergence towards the prescribed tolerance. This behaviour suggests that in the early stages of the Newton-Raphson iterations, the solution is far from the true solution, especially when the mesh is more refined, likely due to higher complexity and initial errors. However, once the solver moves past this initial stagnation, the residuals decrease rapidly, indicating that the solver is functioning as expected and converging quadratically as the iterations progress. This pattern is consistent with the theoretical behaviour of the Newton-Raphson method.

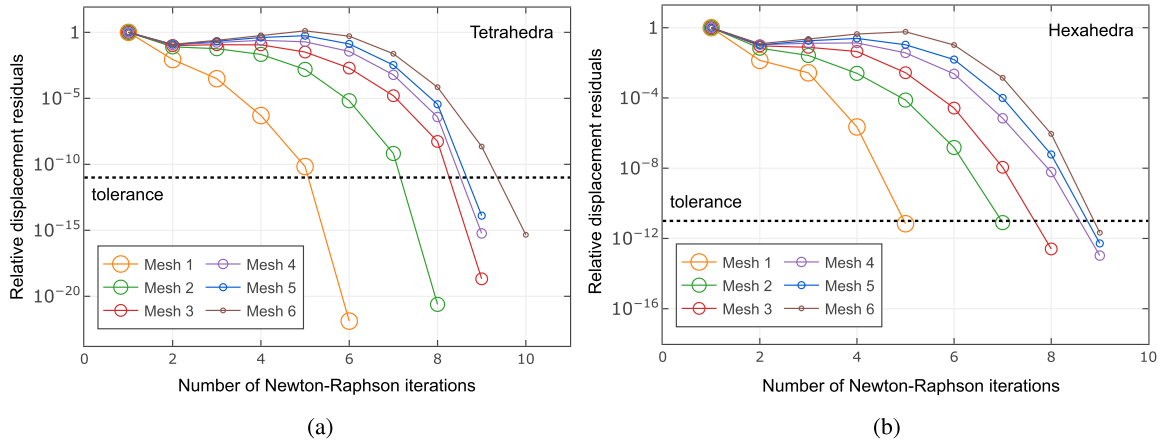


Fig. 14. Narrow T-member. Convergence of the relative displacement residuals of the Newton-Raphson iterations for (a) tetrahedral and (b) hexahedral meshes. The dotted lines represent the prescribed displacement tolerance of 10^{-11} .

5. Conclusion

A vertex-centred block-coupled finite volume methodology has been proposed and assessed for static small-strain elastoplasticity and is made publicly available in the solids4foam toolbox for OpenFOAM. At its core, the methodology relies on a Newton-Raphson algorithm to solve the system of nonlinear equations, where the unknown displacement correction components, located at the primary vertices, are fully coupled and solved simultaneously. Furthermore, the incorporation of the least-squares method for approximating displacement gradients represents an alternative approach compared to conventional shape functions.

This methodology was benchmarked on a range of test cases to assess its accuracy, efficiency and robustness, yielding the following key insights:

- **Patch test.** The patch test confirmed the ability of the coupled solvers to accurately replicate first-order polynomials, showcasing zero discretisation errors under elastic conditions. The analytical and numerical solutions for strains and stresses demonstrated negligible discrepancies, affirming the precision of the developed solver in capturing linear displacement variations.
- **Method of manufactured solutions.** This test case verified the order of convergence of the developed coupled solver. Both tetrahedral and hexahedral mesh configurations exhibited high conformity with analytical solutions for displacement and von Mises stress. The numerical errors, particularly the L_1 and L_2 norms, achieved nearly second-order convergence rates across all variables analysed. These results align closely with theoretical expectations.
- **Perforated plate.** The perforated plate under elastoplastic conditions demonstrated the capability of the coupled solver to handle plastic deformation on a 2-D geometry. The stress and displacement profiles showed good agreement with the cell-centred segregated solver, with second-order convergence observed as the mesh is refined. The computational efficiency of the coupled solver demonstrated significant speedups, achieving up to $7.2\times$ faster performance than the vertex-centred segregated solver and up to $6.0\times$ faster than the cell-centred segregated solver.
- **Narrow T-member.** The narrow T-member test case further tested the efficiency and accuracy of the coupled solver on a 3-D geometry. The solver was evaluated on both tetrahedral and hexahedral meshes, demonstrating that the coupled solver delivered accurate predictions for both displacement and von Mises stress across both mesh types. However, the results exhibit a closer alignment with those obtained from the cell-centred segregated solver when applied to tetrahedral meshes. The computational efficiency of the coupled solver demonstrated significant speedups up to $37.0\times$ faster performance than the vertex-centred segregated solver and up to $27.5\times$ faster than the cell-centred segregated solver. Finally, a convergence analysis was performed on the Newton-Raphson iterations, revealing the presence of a stagnation phase in the relative residuals during the first iterations, but then they rapidly started to decrease, possibly indicating a second-order convergence.

Although the results presented in this work are based on serial simulations to ensure a fair and isolated comparison of algorithmic performance, the developed vertex-centred block-coupled solver has been parallelised by integrating the MPI-based parallelisation strategies of PETSc and OpenFOAM. This enables distributed-memory parallel execution suitable for high-performance computing environments. A comprehensive scalability study is planned for future work, where the solver's strong and weak scaling behaviour will be assessed on multi-core and multi-node systems.

A promising direction for future research involves extending this methodology to large-strain problems using a mixed, total Lagrangian framework, where pressure is introduced as an additional unknown to avoid numerical pressure oscillations that may arise in large isochoric plastic deformations. Additionally, the formulation could be extended to accommodate anisotropic material behaviour by incorporating suitable constitutive laws and updating the consistent tangent operator accordingly. These enhancements would allow the methodology to be applied to a broader class of problems in solid mechanics, including advanced manufacturing simulations

and material characterisation at large deformations. In addition, assessing the impact of the α parameter in the stabilisation term on the accuracy and efficiency of the formulation is left for future work.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used ChatGPT and Grammarly as writing assistants. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Acknowledgements

This publication has emanated from research supported in part by a grant from Science Foundation Ireland under Grant number 16/RC/3872, and industry partner, N.V. Bekaert S.A., Belgium. For the purpose of Open Access, the author has applied a CC BY public copyright licence to any Author Accepted Manuscript version arising from this submission. In addition, the second author gratefully acknowledges financial support from the Irish Research Council (IRC) through the Laureate programme, grant number IRCLA/2017/45.

Data availability

The code and cases used to generate the findings of this study are openly available at <https://doi.org/10.5281/zenodo.15497350>.

References

- [1] P. Cardiff, I. Demirdžić, Thirty years of the finite volume method for solid mechanics, *Arch. Comput. Methods Eng.* 28 (5) (2021) 3721–3780, <https://doi.org/10.1007/s11831-020-09523-0>.
- [2] I. Demirdžić, D. Martinović, A. Ivanković, Numerical simulation of thermal deformation in welded workpiece, *Zavarivanje* 31 (1988) 209–219.
- [3] I. Demirdžić, D. Martinović, Finite volume method for thermo-elasto-plastic stress analysis, *Comput. Methods Appl. Mech. Eng.* 109 (3–4) (1993) 331–349, [https://doi.org/10.1016/0045-7825\(93\)90085-C](https://doi.org/10.1016/0045-7825(93)90085-C).
- [4] I. Demirdžić, S. Muzaferija, Finite volume method for stress analysis in complex domains, *Int. J. Numer. Methods Eng.* 37 (21) (1994) 3751–3766, <https://doi.org/10.1002/nme.1620372110>.
- [5] A. Ivanković, I. Demirdžić, J.G. Williams, P.S. Leewers, Application of the finite volume method to the analysis of dynamic fracture problems, *Int. J. Fract.* 66 (1994) 357–371, <https://doi.org/10.1007/bf00018439>.
- [6] M.A. Wheel, A geometrically versatile finite volume formulation for plane elastostatic stress analysis, *J. Strain Anal.* 31 (2) (1996) 111–116, <https://doi.org/10.1243/03093247V312111>.
- [7] M.A. Wheel, A finite volume method for analysing the bending deformation of thick and thin plates, *Comput. Methods Appl. Mech. Eng.* 147 (1–2) (1997) 199–208, [https://doi.org/10.1016/S0045-7825\(96\)00003-5](https://doi.org/10.1016/S0045-7825(96)00003-5).
- [8] I. Demirdžić, S. Muzaferija, M. Perić, Benchmark solutions of some structural analysis problems using finite volume method and multigrid acceleration, *Int. J. Numer. Methods Eng.* 40 (10) (1997) 1893–1908, [https://doi.org/10.1002/\(SICI\)1097-0207\(19970530\)40:10<1893::AID-NME146>3.0.CO;2-L](https://doi.org/10.1002/(SICI)1097-0207(19970530)40:10<1893::AID-NME146>3.0.CO;2-L).
- [9] A. Ivanković, S. Muzaferija, I. Demirdžić, Finite volume method and multigrid acceleration in modelling of rapid crack propagation in full-scale pipe test, *Comput. Mech.* 20 (1–2) (1997) 46–52, <https://doi.org/10.1007/s004660050215>.
- [10] M.A. Wheel, A mixed finite volume formulation for determining the small strain deformation of incompressible materials, *Int. J. Numer. Methods Eng.* 44 (12) (1999) 1843–1861, [https://doi.org/10.1002/\(SICI\)1097-0207\(19990430\)44:12<1843::AID-NME566>3.0.CO;2-Y](https://doi.org/10.1002/(SICI)1097-0207(19990430)44:12<1843::AID-NME566>3.0.CO;2-Y).
- [11] K. Maneratan, Development of the Finite Volume Method for Non-Linear Structural Applications, Ph.D. thesis, University of London, 2000.
- [12] I. Demirdžić, I. Horman, D. Martinović, Finite volume analysis of stress and deformation in hygro-thermo-elastic orthotropic body, *Comput. Methods Appl. Mech. Eng.* 190 (8–10) (2000) 1221–1232, [https://doi.org/10.1016/S0045-7825\(99\)00476-4](https://doi.org/10.1016/S0045-7825(99)00476-4).
- [13] H. Jasak, H.G. Weller, Application of the finite volume method and unstructured meshes to linear elasticity, *Int. J. Numer. Methods Eng.* 48 (2) (2000) 267–287, [https://doi.org/10.1002/\(SICI\)1097-0207\(20000520\)48:2<267::AID-NME884>3.0.CO;2-Q](https://doi.org/10.1002/(SICI)1097-0207(20000520)48:2<267::AID-NME884>3.0.CO;2-Q).
- [14] I. Bijelonja, I. Demirdžić, S. Muzaferija, A finite volume method for large strain analysis of incompressible hyperelastic materials, *Int. J. Numer. Methods Eng.* 64 (12) (2005) 1594–1609, <https://doi.org/10.1002/nme.1413>.
- [15] H. Bašić, I. Demirdžić, S. Muzaferija, Finite volume method for simulation of extrusion processes, *Int. J. Numer. Methods Eng.* 62 (4) (2005) 475–494, <https://doi.org/10.1002/nme.1168>.
- [16] I. Bijelonja, I. Demirdžić, S. Muzaferija, A finite volume method for incompressible linear elasticity, *Comput. Methods Appl. Mech. Eng.* 195 (44–47) (2006) 6378–6390, <https://doi.org/10.1016/j.cma.2006.01.005>.
- [17] v. Tuković, H. Jasak, Updated Lagrangian finite volume solver for large deformation dynamic response of elastic body, *Trans. FAMENA* 31 (1) (2007) 55–70.
- [18] S. Das, S.R. Mathur, J.Y. Murthy, An unstructured finite-volume method for structure–electrostatics interactions in MEMS, *Numer. Heat Transf., Part B, Fundam.* 60 (6) (2011) 425–451, <https://doi.org/10.1080/10407790.2011.628252>.
- [19] S. Das, S.R. Mathur, J.Y. Murthy, Finite-volume method for creep analysis of thin RF MEMS devices using the theory of plates, *Numer. Heat Transf., Part B, Fundam.* 61 (2) (2012) 71–90, <https://doi.org/10.1080/10407790.2012.646170>.
- [20] v. Tuković, A. Ivanković, A. Karač, Finite-volume stress analysis in multi-material linear elastic body, *Int. J. Numer. Methods Eng.* 93 (4) (2012) 400–419, <https://doi.org/10.1002/nme.4390>.
- [21] P. Cardiff, A. Karač, A. Ivanković, A large strain finite volume method for orthotropic bodies with general material orientations, *Comput. Methods Appl. Mech. Eng.* 268 (2014) 318–335, <https://doi.org/10.1016/j.cma.2013.09.008>.
- [22] T. Tang, O. Heddal, P. Cardiff, On finite volume method implementation of poro-elasto-plasticity soil model, *Int. J. Numer. Anal. Methods Geomech.* 39 (13) (2015) 1410–1430, <https://doi.org/10.1002/nag.2361>.
- [23] I. Demirdžić, A fourth-order finite volume method for structural analysis, *Appl. Math. Model.* 40 (4) (2016) 3104–3114, <https://doi.org/10.1016/j.apm.2015.09.098>.
- [24] P. Cardiff, v. Tuković, H. Jasak, A. Ivanković, A block-coupled finite volume methodology for linear elasticity and unstructured meshes, *Comput. Struct.* 175 (2016) 100–122, <https://doi.org/10.1016/j.compstruc.2016.07.004>.
- [25] P. Cardiff, v. Tuković, P.D. Jaeger, M. Clancy, A. Ivanković, A Lagrangian cell-centred finite volume method for metal forming simulation, *Int. J. Numer. Methods Eng.* 109 (13) (2017) 1777–1803, <https://doi.org/10.1002/nme.5345>.

- [26] I. Bijelonja, I. Demirdžić, S. Muzaferija, Mixed finite volume method for linear thermoelasticity at all Poisson's ratios, *Numer. Heat Transf., Part A, Appl.* 72 (3) (2017) 215–235, <https://doi.org/10.1080/10407782.2017.1372665>.
- [27] A. Golubović, I. Demirdžić, S. Muzaferija, Finite volume analysis of laminated composite plates, *Int. J. Numer. Methods Eng.* 109 (11) (2017) 1607–1620, <https://doi.org/10.1002/nme.5347>.
- [28] I. González Acedo, A. Naseri, J. Chiva Segura, J. Rigola Serrano, C.D. Pérez Segarra, An enhanced finite volume based solver for thermoelastic materials in fluid-structure coupled problems, in: 6th European Conference on Computational Mechanics, 2018, pp. 1361–1372.
- [29] I. González Acedo, Development of a Finite Volume Method for Elastic Materials and Fluid-Solid Coupled Applications, Ph.D. thesis, Universitat Politècnica de Catalunya, 2019.
- [30] M. Clancy, Improving the Predictive Capabilities of Wire Processing Simulations, Ph.D. thesis, University College Dublin, 2019.
- [31] L. Ribeiro de Azevedo, Development of a Block-Coupled Finite Volume Methodology for Non-Linear Elasticity, Ph.D. thesis, Technological University Darmstadt, 2020.
- [32] K. Wu, v. Tuković, P. Cardiff, A. Ivanković, Hierarchical RVE-based multiscale modelling of non-linear heterogeneous materials using the finite volume method, *Int. J. Multiscale Comput. Eng.* 20 (2021) 83–103, <https://doi.org/10.1615/IntJMultCompEng.2022040214>.
- [33] A. Scolaro, C. Fiorina, I. Clifford, A. Pautz, Development of a semi-implicit contact methodology for finite volume stress solvers, *Int. J. Numer. Methods Eng.* 123 (2) (2022) 309–338, <https://doi.org/10.1002/nme.6857>.
- [34] W. Boscheri, S. Chicchetti, I. Peshkov, A cell-centered implicit-explicit Lagrangian scheme for a unified model of nonlinear continuum mechanics on unstructured meshes, *J. Comput. Phys.* 451 (2022) 1–44, <https://doi.org/10.1016/j.jcp.2021.110852>.
- [35] S. Bali, v. Tuković, P. Cardiff, A. Ivanković, V. Pakrashi, A cell-centered finite volume formulation of geometrically exact simo-Reissner beams with arbitrary initial curvatures, *Int. J. Numer. Methods Eng.* 123 (17) (2022) 3950–3973, <https://doi.org/10.1002/nme.6994>.
- [36] I. Batistić, P. Cardiff, A. Ivanković, v. Tuković, A finite volume penalty-based implicit procedure for the treatment of the frictionless contact boundaries, *Int. J. Numer. Methods Eng.* 124 (18) (2023) 4171–4191, <https://doi.org/10.1002/nme.7302>.
- [37] P. Castrillo Green, High-Order Finite Volume Method for Solid Dynamics in Fluid-Structure Interaction Applications, Ph.D. thesis, Universitat Politècnica de Catalunya, 2023.
- [38] A. Whelan, Finite Volume Implementation of Damage and Fracture Mechanics and Prediction of Damage in Metal Forming Processes, Ph.D. thesis, University College Dublin, 2023.
- [39] Y.D. Fryer, C. Bailey, M. Cross, C.H. Lai, A control volume procedure for solving the elastic stress-strain equations on an unstructured mesh, *Appl. Math. Model.* 15 (11–12) (1991) 639–645, [https://doi.org/10.1016/S0307-904X\(99\)81010-X](https://doi.org/10.1016/S0307-904X(99)81010-X).
- [40] G.A. Taylor, C. Bailey, M. Cross, Solution of the elastic/visco-plastic constitutive equations: a finite volume approach, *Appl. Math. Model.* 19 (12) (1995) 746–760, [https://doi.org/10.1016/0307-904X\(95\)00093-y](https://doi.org/10.1016/0307-904X(95)00093-y).
- [41] C. Bailey, M. Cross, A finite volume procedure to solve elastic solid mechanics problems in three dimensions on an unstructured mesh, *Int. J. Numer. Methods Eng.* 38 (10) (1995) 1757–1776, <https://doi.org/10.1002/nme.1620381010>.
- [42] G.A. Taylor, A Vertex Based Discretisation Scheme Applied to Material Non-Linearity Within a Multi-Physics Finite Volume Framework, Ph.D. thesis, University of Greenwich, 1996.
- [43] N.A. Fallah, C. Bailey, M. Cross, G.A. Taylor, Comparison of finite element and finite volume methods application in geometrically nonlinear stress analysis, *Appl. Math. Model.* 24 (7) (2000) 439–455, [https://doi.org/10.1016/S0307-904X\(99\)00047-5](https://doi.org/10.1016/S0307-904X(99)00047-5).
- [44] N.A. Fallah, Computational Stress Analysis Using Finite Volume Methods, Ph.D. thesis, University of Greenwich, 1996.
- [45] A.K. Slone, K. Pericleous, C. Bailey, M. Cross, Dynamic fluid-structure interaction using finite volume unstructured mesh procedures, *Comput. Struct.* 80 (5–6) (2002) 371–390, [https://doi.org/10.1016/S0045-7949\(01\)00177-8](https://doi.org/10.1016/S0045-7949(01)00177-8).
- [46] A.K. Slone, C. Bailey, M. Cross, Dynamic solid mechanics using finite volume methods, *Appl. Math. Model.* 27 (2) (2003) 69–87, [https://doi.org/10.1016/S0307-904X\(02\)00060-4](https://doi.org/10.1016/S0307-904X(02)00060-4).
- [47] G.A. Taylor, C. Bailey, M. Cross, A vertex-based finite volume method applied to non-linear material problems in computational solid mechanics, *Int. J. Numer. Methods Eng.* 56 (4) (2003) 507–529, <https://doi.org/10.1002/nme.574>.
- [48] A.K. Slone, T.N. Croft, A.J. Williams, M. Cross, An alternative mixed Eulerian Lagrangian approach to high speed collision between solid structures on parallel clusters, *Adv. Eng. Softw.* 38 (4) (2007) 244–255, <https://doi.org/10.1016/j.advengsoft.2006.09.015>.
- [49] A.J. Williams, A.K. Slone, T.N. Croft, M. Cross, A mixed Eulerian-Lagrangian method for modelling metal extrusion processes, *Comput. Methods Appl. Mech. Eng.* 199 (33–36) (2010) 2123–2134, <https://doi.org/10.1016/j.cma.2010.03.001>.
- [50] I.O.I. Hassan, A Vertex Centred Finite Volume Algorithm for Fast Solid Dynamics: Total and Updated Lagrangian Descriptions, Ph.D. thesis, 2018.
- [51] P.V. de Cassia Lima Pimenta, F. Marcondes, Two-dimensional rate-independent plasticity using the element-based finite volume method, *J. Braz. Soc. Mech. Sci. Eng.* 41 (142) (2019) 1–16, <https://doi.org/10.1007/s40430-019-1641-y>.
- [52] V. Cardoso Nungaray, S. Botello, F. Zarate, E. Oñate, Finite volume for stress analysis upon polygonal, unstructured and non conforming meshes, *Rev. Int. Métodos Numér. Cálculo Diseño Ing.* 35 (4) (2019) 1–11, <https://doi.org/10.23967/j.rimni.2019.10.001>.
- [53] L.M. Vieira, M. Giacomini, R. Sevilla, A. Huerta, A second-order face-centred finite volume method for elliptic problems, *Comput. Methods Appl. Mech. Eng.* 358 (2020), <https://doi.org/10.1016/j.cma.2019.112655>.
- [54] M. Perić, R. Kessler, G. Scheurer, Comparison of finite-volume numerical methods with staggered and colocated grids, *Comput. Fluids* 16 (4) (1988) 389–403, [https://doi.org/10.1016/0045-7930\(88\)90024-2](https://doi.org/10.1016/0045-7930(88)90024-2).
- [55] M. Aguirre, A.J. Gil, J. Bonet, A.A. Carreno, A vertex centred finite volume Jameson-Schmidt-Turkel (JST) algorithm for a mixed conservation formulation in solid dynamics, *J. Comput. Phys.* 259 (2014) 672–699, <https://doi.org/10.1016/j.jcp.2013.12.012>.
- [56] M. Aguirre, A.J. Gil, J. Bonet, C.H. Lee, An upwind vertex centred finite volume solver for Lagrangian solid dynamics, *J. Comput. Phys.* 300 (2015) 387–422, <https://doi.org/10.1016/j.jcp.2015.07.029>.
- [57] R. Costa, S. Clain, G.J. Machado, New cell-vertex reconstruction for finite volume scheme: application to the convection–diffusion–reaction equation, *Comput. Math. Appl.* 68 (10) (2014) 1229–1249, <https://doi.org/10.1016/j.camwa.2014.08.015>.
- [58] A. Whelan, T. Tang, V. Pakrashi, P. Cardiff, A finite volume framework for damage and fracture prediction in wire drawing, *Int. J. Numer. Methods Eng.* 126 (2025) e7640, <https://doi.org/10.1002/nme.7640>.
- [59] J.C. Simo, T.J. Hughes, *Computational Inelasticity*, Springer, 1998.
- [60] J. Blazek, *Computational Fluid Dynamics: Principles and Applications*, Butterworth-Heinemann, 2015.
- [61] D.J. Mavriplis, Unstructured grid techniques, *Annu. Rev. Fluid Mech.* 29 (1) (1997) 473–514, <https://doi.org/10.1146/annurev.fluid.29.1.473>.
- [62] K.-J. Bathe, *Finite Element Procedures*, 1st edition, Prentice Hall, 1996.
- [63] M. Vaz, P. Muñoz-Rojas, G. Filippini, On the accuracy of nodal stress computation in plane elasticity using finite volumes and finite elements, *Comput. Struct.* 87 (17) (2009) 1044–1057, <https://doi.org/10.1016/j.compstruc.2009.05.007>.
- [64] O.C. Zienkiewicz, R.L. Taylor, *The Finite Element Method*, vol. 1, Butterworth-Heinemann, 2000.
- [65] S. Balay, W.D. Gropp, L.C. McInnes, B.F. Smith, Efficient management of parallelism in object oriented numerical software libraries, in: E. Arge, A.M. Bruaset, H.P. Langtangen (Eds.), *Modern Software Tools in Scientific Computing*, Birkhäuser Press, 1997, pp. 163–202.
- [66] W. Arendt, M. Warma, Dirichlet and Neumann boundary conditions: what is in between?, in: *Nonlinear Evolution Equations and Related Topics*, Birkhäuser, Basel, 2003, pp. 119–135.

- [67] L. Demkowicz, A note on symmetry boundary conditions in finite element methods, *Appl. Math. Lett.* 4 (5) (1991) 27–30, [https://doi.org/10.1016/0893-9659\(91\)90138-L](https://doi.org/10.1016/0893-9659(91)90138-L).
- [68] G.P. Bazeley, Y.K. Cheung, B.M. Irons, O.C. Zienkiewicz, Triangular elements in plate bending, in: *Proceedings of First Conference on Matrix Methods in Structural Mechanics*, 1965.
- [69] T. Belytschko, W.K. Liu, B. Moran, K.I. Elkhodary, *Nonlinear Finite Elements for Continua and Structures*, 2nd edition, Wiley, 2014.
- [70] P. Cardiff, A. Karac, P.D. Jaeger, H. Jasak, J. Nagy, A. Ivanković, Ž. Tuković, An open-source finite volume toolbox for solid mechanics and fluid-solid interaction simulations, <https://doi.org/10.48550/arXiv.1808.10736>, 2018.
- [71] Ž. Tuković, A. Karač, P. Cardiff, H. Jasak, A. Ivanković, OpenFOAM finite volume solver for fluid-solid interaction, *Trans. FAMENA* 42 (3) (2018) 1–31, <https://doi.org/10.21278/TOF.42301>.
- [72] C. Geuzaine, J.-F. Remacle, Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities, *Int. J. Numer. Methods Eng.* 79 (11) (2009) 1309–1331, <https://doi.org/10.1002/nme.2579>.